

Fraud Detection Capstone Project

Complete implementation study guide

From the original notebook to the reusable Python pipeline, saved model, and single-transaction prediction.

Study scope	Verified project facts
Dataset	11,142 transactions • 10 source columns
Learning task	Binary classification: legitimate (0) or fraud (1)
Models	Logistic Regression, Random Forest, Gradient Boosting
Saved predictor	Random Forest with 100 trees and 14 input features
Guide contents	27 sections • 90 viva questions • 2-page final cheat sheet

Based on the project files inspected on 23 September 2026.

Reading rule: “Project evidence” refers to the supplied notebook, dataset, source code, or saved artifact. “Concept” explains the mathematics or programming behind that evidence. Proposed Streamlit behavior is explicitly marked as future work. No original project source or data was edited to produce this guide.

All amounts use the dataset’s units. The notebook displays the rupee symbol, but currency provenance is not established by the supplied workbook.

Contents

01 Project overview	3
02 Dataset explanation	4
03 EDA: exploratory data analysis	6
04 Feature engineering	10
05 Data preprocessing	12
06 Train/test split	14
07 Logistic Regression	15
08 Random Forest	16
09 Gradient Boosting	18
10 Model evaluation	19
11 Model comparison	22
12 Feature importance	23
13 Financial impact analysis	25
14 Business questions	26
15 Python project structure	27
16 preprocessing.py: code walkthrough	28
17 train.py: code walkthrough	31
18 predict.py: code walkthrough	36
19 Joblib and model saving	38
20 requirements.txt	39
21 End-to-end data flow	40
22 Reproducibility	42
23 Current limitations	43
24 Streamlit: preparation for the next phase	44
25 Viva preparation: 90 questions	45
26 Quick revision	60
27 Final project cheat sheet	64

01 Project overview

Evidence: Notebook cells 1-2; src/train.py; src/predict.py; saved model artifact.

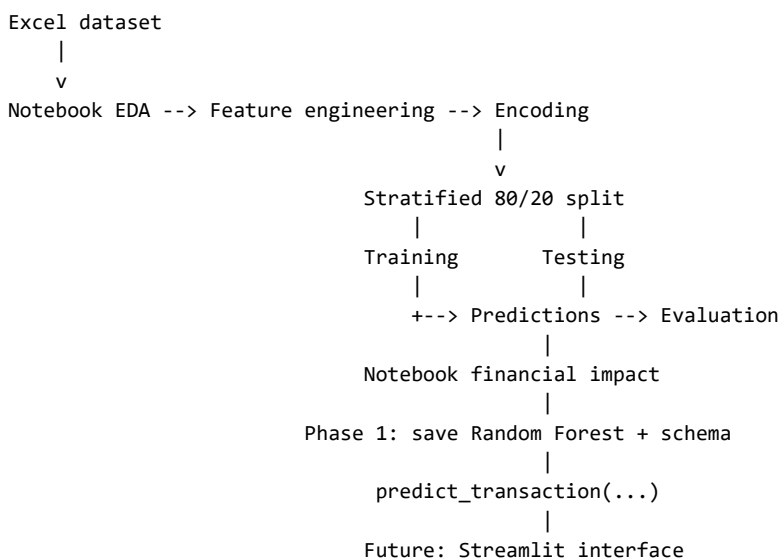
What problem does this project solve?

The project studies financial transaction records and learns a rule for separating records labelled legitimate from records labelled fraudulent. A transaction is one row. The model reads facts about that row, such as its type, amount, time step, and account balances. It produces a predicted class and, for the saved Random Forest, a model-estimated fraud probability.

This is supervised binary classification. “Supervised” means the training examples already contain the correct label. “Binary” means there are two possible classes. The target column is **isFraud: 0 means legitimate** and **1 means fraudulent**. Fraud detection here means learning to reproduce these provided labels; the files do not describe the investigation or labelling procedure that originally assigned them.

The workflow and the implementation boundary

Stage	What actually exists
1. Dataset → EDA	The Excel workbook and notebook contain inspection, tables, and plots.
2. Feature engineering → Encoding	Notebook and preprocessing.py calculate four balance features and dummy-encode type.
3. Split → Train → Predict → Evaluate	Notebook and train.py compare three models on the same stratified test set.
4. Financial impact	Notebook calculates fraud amounts and Random Forest error amounts. train.py does not reproduce this business-reporting block.
5. Saved model → New predictions	Phase 1 saves the Random Forest and schema; predict.py loads them for inference.
6. Streamlit application	Future phase only. No app, UI, or Streamlit dependency is present.



How to study this guide

First understand the data and four formulas in sections 2-6. Then study the three model families and evaluation in sections 7-14. Read sections 15-22 with the Python files open. Use section 25 to practise speaking answers, then use the final two pages for immediate revision. You do not need to memorise every decimal; you should understand what each denominator represents.

02 Dataset explanation

Evidence: Fraud_Analysis_Dataset.xlsx, sheet in, A1:J11143; notebook cells 3-17.

The workbook is **Fraud_Analysis_Dataset.xlsx**. It contains one worksheet named **in**, one header row, and **11,142 data rows**. There are **10 original columns**. The original notebook loads a Downloads-folder copy through an absolute path. Phase 1 loads the workbook beside the project's src directory through a portable path.

Data dictionary

Column	pandas dtype	Meaning
step	int64	Simulated time unit. The notebook states one step equals one hour; no calendar timestamps are supplied.
type	object	Transaction category: CASH_IN, CASH_OUT, DEBIT, PAYMENT, or TRANSFER.
amount	float64	Recorded amount of this transaction, in the dataset's monetary units.
nameOrig	object	Identifier for the originating account. It is not an amount and is not a model input.
oldbalanceOrg	float64	Origin account balance before the transaction. Preserve the exact Org spelling.
newbalanceOrig	float64	Origin account balance after the transaction; the name uses Orig.
nameDest	object	Identifier for the destination account. Excluded from model inputs.
oldbalanceDest	float64	Destination balance before the transaction.
newbalanceDest	float64	Destination balance after the transaction.
isFraud	int64	Provided target label: 0 legitimate, 1 fraud. Never passed to the model as a feature.

These are the pandas types reported after Excel loading. int64 represents whole numbers, float64 represents double-precision numbers with decimal support, and object holds the text values in this dataset. The account-field meanings follow the notebook's usage and column names. No separate supplier data dictionary or dataset provenance document is included.

Class balance and transaction categories

Label	Count	Share
0: legitimate	10,000	89.75%
1: fraud	1,142	10.25%
Total	11,142	100%

The exact fraud fraction is $1,142 / 11,142$; 10.25% is its rounded percentage. There are many more legitimate records than fraudulent records, so accuracy alone would not describe how well the model finds fraud.

Type	Legitimate	Fraud	Fraud rate
CASH_IN	1951	0	0.00%
CASH_OUT	1293	578	30.89%
DEBIT	346	0	0.00%
PAYMENT	5510	0	0.00%
TRANSFER	900	564	38.52%

In ordinary transaction terminology CASH_IN adds funds, CASH_OUT withdraws funds, PAYMENT pays a recipient, TRANSFER moves funds between accounts, and DEBIT records a debit transaction. These are plain-language readings of the category labels; detailed processing rules for each category are not supplied. The observed zero fraud counts for three categories describe this sample, not a guarantee that those transaction types can never be fraudulent.

Data quality checks

Missing values: zero in every column. **Duplicate rows:** zero exact duplicate rows. The notebook therefore has no missing-value imputation or duplicate-removal step. Repeated amounts, repeated account IDs, or similar transactions are not the same as a duplicated full row. The duplicate check does not establish that each account is unique.

03 EDA: exploratory data analysis

Evidence: Notebook cells 3-36. All operations below already appear in the notebook.

EDA means examining data before fitting a model. It helps answer: what is in the table, are values missing, how common is fraud, and which patterns may matter? It is an investigation stage, not a model that predicts fraud. The notebook performs EDA on the complete dataset; its reported descriptive findings therefore use all 11,142 rows.

Loading and dimensions

```
pd.read_excel(...); df.shape
```

What it does and what was observed: `read_excel` converts the workbook's first sheet into a DataFrame. `shape` is a tuple: number of rows, then columns. The result (11142, 10) establishes the size of the source table.

Why it matters: Without checking shape, a wrong worksheet or incomplete file could be mistaken for the intended dataset.

First records

```
df.head()
```

What it does and what was observed: `head` displays the first five rows by default. The notebook's first records are labelled fraud and include paired TRANSFER and CASH_OUT amounts. This is a preview, not a random sample or proof that the whole dataset is mostly fraudulent.

Why it matters: It helps confirm field names and plausible loading before later calculations.

Structure and types

```
df.info()
```

What it does and what was observed: `info` prints column names, non-null counts, inferred dtypes, and a memory estimate. It reports two integer columns, five floating-point columns, and three object columns. All ten columns have 11,142 non-null entries.

Why it matters: The string-valued type column needs encoding; balance and amount columns are already numeric.

Numerical summary

```
df.describe().T
```

What it does and what was observed: `describe` calculates count, mean, standard deviation, minimum, quartiles, and maximum for numeric columns. `T` transposes the result so each original column becomes a row.

Why it matters: The amount distribution has a large upper tail; the mean and median differ substantially.

Missing values

```
df.isnull().sum().sort_values(ascending=False)
```

What it does and what was observed: `isnull` produces a Boolean table. `sum` counts True values column by column. `sort_values` puts the largest counts first. Every count is zero.

Why it matters: No imputation was necessary for this workbook. The check does not prove every non-null value is semantically correct.

Exact duplicate rows

```
df.duplicated().sum()
```

What it does and what was observed: `duplicated` marks repeated complete rows after their first occurrence. Summing gives zero.

Why it matters: There is no duplicate-removal operation because this check found none.

Label counts and percentages

```
df["isFraud"].value_counts(); value_counts(normalize=True)
```

What it does and what was observed: `value_counts` counts each label; `normalize=True` divides by total rows. `sort_index` orders labels as 0 then 1; multiplying by 100 produces percentages.

Why it matters: 10,000 legitimate and 1,142 fraudulent records demonstrate the class imbalance.

Target plot

```
sns.countplot(data=df, x="isFraud")
```

What it does and what was observed: The count plot draws one bar per class. `ax.bar_label` adds the counts. Titles and axis labels explain the class coding.

Why it matters: It visually confirms that legitimate transactions are much more numerous.

Type by label

```
pd.crosstab(df["type"], df["isFraud"])
```

What it does and what was observed: `crosstab` counts every combination of transaction type and label. It is a contingency table, not a correlation matrix.

Why it matters: All 1,142 fraudulent records are in TRANSFER or CASH_OUT; their counts are 564 and 578.

Within-type fraud rate

```
df.groupby("type")["isFraud"].mean().mul(100)
```

What it does and what was observed: `groupby` separates rows by category. A mean of 0/1 labels equals the fraction labelled 1. Sorting descending orders the rates.

Why it matters: TRANSFER has 38.52% fraud and CASH_OUT 30.89%. The denominator is all transactions of that type, not all fraud records.

Fraud-rate bars

```
fraud_rate_by_type.plot(kind="bar")
```

What it does and what was observed: A bar chart compares category-specific rates. A later notebook section repeats the calculation as fractions and renames count-table columns Legitimate and Fraud.

Why it matters: 38.52% and approximately 0.3852 describe the same rate with different units; repeated cells are not a new experiment.

Amount by label

```
df.groupby("isFraud")["amount"].describe()
```

What it does and what was observed: This generates separate amount summaries for legitimate and fraudulent records.

Why it matters: Fraud-labelled rows have higher mean and median amounts in this sample; amount alone is not a perfect classifier.

Amount boxplot

```
sns.boxplot(data=df, x="isFraud", y="amount")
```

What it does and what was observed: A boxplot shows the median, interquartile range, whiskers, and points outside the whiskers. The notebook limits the visible y-axis to the 99th percentile of amount.

Why it matters: The axis limit improves visibility. It does not delete or cap any data used by training.

Step investigation

```
pd.crosstab(df["step"], df["isFraud"]); groupby(...).agg(...)
```

What it does and what was observed: The crosstab counts records by simulated hour and label; agg calculates minimum, maximum, mean, and median. A stacked sns.histplot uses 30 bins and hue=isFraud.

Why it matters: Legitimate records stop at step 7 but fraud extends to step 95. The notebook flags a possible sampling or temporal problem.

Engineered-feature summaries

```
df.groupby("isFraud")[engineered_features].mean(); .median()
```

What it does and what was observed: After engineering features, the notebook compares group means and medians for amount and all four derived balance quantities.

Why it matters: The fraud-group median orig_balance_error is 0.00; the legitimate-group median is 5,573.72. These are patterns, not decision rules.

Fraud record inspection

```
fraud_df = df[df["isFraud"] == 1]; fraud_df[...].head(20)
```

What it does and what was observed: Boolean filtering selects labelled fraud. Selected columns display balances, amounts, and changes for the first 20 such rows.

Why it matters: This illustrates individual balance patterns, including exceptions; it does not evaluate a model.

Type, amount, and class together

```
sns.boxplot(data=df, x="type", y="amount", hue="isFraud")
```

What it does and what was observed: hue separates boxplots by class inside transaction type. This plot also limits its visible y-axis using the 99th percentile.

Why it matters: It compares amount distributions within transaction types rather than only across all rows.

Numeric correlation heatmap

```
corr = df[numeric_cols].corr(); sns.heatmap(corr, annot=True, fmt=".2f")
```

What it does and what was observed: corr uses Pearson correlation by default on the chosen numeric columns. The heatmap annotates rounded values and centres its colour scale at zero.

Why it matters: It describes linear association among balances, engineered features, step, amount, and isFraud. Account IDs and unencoded type are excluded from this matrix.

Reading the amount statistics

Statistic	Legitimate	Fraud
Count	10,000	1,142
Mean	101,339.73	1,192,628.93
Median	12,725.24	353,179.45
Minimum	2.39	119.00
Maximum	3,776,389.09	10,000,000.00

The mean adds the amounts and divides by the count; large values pull it upward. The median is the middle value after sorting and is less affected by a few very large transactions. Standard deviation measures spread around the mean. The 25th and 75th percentiles bound the middle half of the data. These summaries motivate inspection; they do not justify silently removing outliers.

Reading the correlation calculation

```
Pearson r = sum((x - mean(x)) * (z - mean(z)))
            / sqrt(sum((x - mean(x))^2) * sum((z - mean(z))^2))
```

Here x and z are two numeric columns evaluated over the same rows. Values near +1 indicate strong positive linear association; values near -1 indicate strong negative linear association; values near 0 indicate weak linear association. A near-zero correlation does not rule out nonlinear patterns or interactions. A high correlation is not proof of causation.

The following values are the same notebook correlation calculation recomputed from the unchanged workbook for readability, rounded to four decimals. They are not new predictive experiments.

Feature	Correlation with isFraud
step	0.8320
amount	0.4355
oldbalanceOrg	0.0464
newbalanceOrig	-0.1279
oldbalanceDest	-0.0566

Feature	Correlation with isFraud
newbalanceDest	-0.0029
orig_balance_change	0.4872
dest_balance_change	0.1240
orig_balance_error	-0.1412
dest_balance_error	0.1780

What the EDA does not establish

No separate EDA for an external dataset, cross-validation analysis, SHAP analysis, geographic study, or account-network analysis appears in the project. The notebook raises the step concern but never resolves it. Its charts and descriptive results should be discussed as observations on this supplied sample.

04 Feature engineering

Evidence: Notebook cells 24-29; src/preprocessing.py, engineer_features().

Feature engineering transforms existing inputs into additional quantities that may make patterns easier for a model to recognise. This project adds exactly four features. None uses isFraud. Each feature is computed independently for a row, so no dataset-wide mean, fitted scaler, or imputation statistic is required.

orig_balance_change

```
orig_balance_change = oldbalanceOrig - newbalanceOrig
```

Variables: oldbalanceOrig is the origin balance before the transaction; newbalanceOrig is the origin balance after it.

Interpretation: This is the recorded reduction in the origin account balance. A positive value means the origin balance decreased; zero means it stayed the same; a negative value means it increased. The direction is deliberately before minus after.

Purpose and actual example: The model can compare account depletion with the stated amount. In the actual first row, $181.00 - 0.00 = 181.00$.

dest_balance_change

```
dest_balance_change = newbalanceDest - oldbalanceDest
```

Variables: newbalanceDest is the destination balance after the transaction; oldbalanceDest is its balance before it.

Interpretation: This is the recorded increase in the destination balance. Positive means the destination balance increased, zero means no recorded change, and negative means it decreased. The direction differs from the origin formula.

Purpose and actual example: It measures the receiving-side balance movement. In the actual first row, $0.00 - 0.00 = 0.00$.

orig_balance_error

$\text{orig_balance_error} = \text{amount} - \text{orig_balance_change}$

Variables: amount is the stated transaction amount; orig_balance_change is the origin decrease calculated above.

Interpretation: This is the signed difference between the transaction amount and the recorded origin decrease. A zero means the two numbers agree. Positive means amount exceeds that decrease; negative means the decrease exceeds amount.

Purpose and actual example: It makes a possible balance discrepancy explicit. In the first row, $181.00 - 181.00 = 0.00$.

dest_balance_error

$\text{dest_balance_error} = \text{amount} - \text{dest_balance_change}$

Variables: amount is the stated transaction amount; dest_balance_change is the recorded destination increase.

Interpretation: This compares the stated amount with the destination increase. A zero means agreement; positive means amount exceeds the recorded increase; negative means the increase exceeds amount.

Purpose and actual example: In the first row, $181.00 - 0.00 = 181.00$. The destination did not show the same increase as the stated transfer amount.

Why “error” does not automatically mean fraud

These names describe arithmetic residuals, not confirmed data mistakes or legal findings. The code applies the same formula to every transaction type. For example, a cash-in can increase the origin balance, making orig_balance_change negative. A large residual may reflect transaction conventions or recorded balances; it is simply another input to a classifier. The project does not implement a rule saying any nonzero error is fraud.

The engineered features are algebraically related to their source columns. Adding them supplies convenient representations of existing information, not four independent new measurements. Tree models may split efficiently on these differences. Logistic Regression receives related columns without scaling, exactly as in the notebook.

Feature median	Legitimate	Fraud
orig_balance_change	538.59	348,705.15
dest_balance_change	0.00	3,733.04
orig_balance_error	5,573.72	0.00
dest_balance_error	6,605.92	9,439.94

05 Data preprocessing

Evidence: Notebook cells 38-39; preprocessing.py; saved artifact preprocessing dictionary.

Inputs, identifiers, and the target

The source table includes ten columns, but only seven raw fields are needed for prediction: step, amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest, and type. The account IDs nameOrig and nameDest are not selected by the feature list. isFraud is copied into y and excluded from X. Including the answer in X would let the model read the label rather than learn from legitimate inputs.

An account identifier is a name for an entity, not an ordered measurement. A larger number inside an ID does not imply more risk. The notebook specifically cautions against blindly using nameOrig numerically; its feature list excludes both account identifiers. No account aggregation or identity encoding is implemented.

Why encode transaction type?

The estimators operate on numeric feature arrays. A string such as TRANSFER has no directly usable arithmetic value. One-hot encoding represents membership in categories as separate indicator columns. The implementation uses pandas get_dummies, with drop_first=True, and does not assign arbitrary ordered values such as PAYMENT=1 and TRANSFER=2.

```
X = df[features].copy()
y = df["isFraud"].copy()
X = pd.get_dummies(X, columns=["type"], drop_first=True)
```

There are five categories. The stored order is CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER. Dropping the first category leaves four Boolean dummy columns; CASH_IN is the reference category represented by all four indicators being False. With an intercept, dropping one indicator removes the redundant all-categories sum relationship. It does not drop CASH_IN transactions.

Type	CASH_OUT	DEBIT	PAYMENT	TRANSFER
CASH_IN	0	0	0	0
CASH_OUT	1	0	0	0
DEBIT	0	1	0	0
PAYMENT	0	0	1	0
TRANSFER	0	0	0	1

The table abbreviates the actual column names, which start with type_. True and False behave as numeric membership indicators. The encoded matrix contains **14 features**: six original numeric columns, four engineered numeric columns, and four dummy columns.

Order	Actual feature name
1	step
2	amount

Order	Actual feature name
3	oldbalanceOrg
4	newbalanceOrig
5	oldbalanceDest
6	newbalanceDest
7	orig_balance_change
8	dest_balance_change
9	orig_balance_error
10	dest_balance_error
11	type_CASH_OUT
12	type_DEBIT
13	type_PAYMENT
14	type_TRANSFER

The six X/y names

Variable	Meaning	Shape
X	All model inputs after feature engineering and encoding	11,142 × 14
y	All known fraud labels, aligned to X by row	11,142 labels
X_train	Inputs used to fit each model	8,913 × 14
X_test	Held-out inputs used to evaluate models	2,229 × 14
y_train	Known labels for training inputs	8,913 labels
y_test	Known labels for held-out inputs	2,229 labels

A DataFrame is a labelled two-dimensional table; a Series is one labelled column. `copy()` gives a separate object so feature preparation does not intentionally overwrite the caller's source table. This project's "pipeline" is a sequence of reusable functions; it is not a scikit-learn Pipeline object.

Training versus prediction encoding

Training learns the category list from the complete dataset before the split, preserving the notebook order of operations. Prediction explicitly sets a pandas Categorical with those saved categories before `get_dummies`. Otherwise encoding a single TRANSFER row with `drop_first=True` could remove its only observed category and produce no type dummy columns. `reindex` then enforces the exact saved feature order. Unknown or missing transaction types are rejected.

06 Train/test split

Evidence: Notebook cell 40; train.py lines 28-30; reconstructed split from unchanged data.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

Training data teach the model. Test data ask how the fitted model performs on rows it did not fit on. A useful analogy is practising with one set of exam questions and then taking a separate exam. Seeing the exam's answers while preparing would make the test less informative. Here each of the three models receives the same practice set and the same test set.

The 80/20 split

test_size=0.20 reserves approximately one fifth of rows for testing. The row count cannot be fractional, so the actual split is 8,913 training transactions and 2,229 test transactions. train_test_split shuffles by default. It splits X and y together to preserve which label belongs to which transaction.

Partition	Legitimate	Fraud	Total
Train	7999	914	8913
Test	2001	228	2229
All	10000	1142	11142

Stratification

stratify=y asks the splitter to preserve the class proportions approximately in both partitions. It does not create synthetic transactions, balance the classes to 50/50, or give fraud cases extra model weight. Fraud remains the minority class. Exact percentages differ slightly because counts must be whole numbers.

Random state and reproducibility

random_state=42 fixes the pseudo-random seed used by this split. With the same input ordering and compatible implementation, the same rows are assigned to training and testing. The number 42 has no fraud-specific meaning and is not a performance score. A different seed could change which examples are held out and therefore change results.

Reproducibility means another run can follow the same procedure and obtain the same outputs within stated precision. A fixed split seed is one ingredient; package versions, data values, row order, feature order, and model parameters also matter. It does not make an evaluation representative of all future transactions.

What this split does not do

This is a random stratified holdout, not a time-ordered split, group-by-account split, or cross-validation. The project does not report any such alternative evaluation. In particular, preserving class proportions does not solve the notebook's unusual step distribution.

07 Logistic Regression

Evidence: Notebook cell 41; train.py; artifact model_parameters and evaluations.

Concept: Logistic Regression is a classification model despite its name. It combines numeric features into a score and converts that score to a number between zero and one. For this binary task, that number estimates the probability of class 1 under the fitted model. A class decision then turns the score into either 0 or 1.

$$z = b + w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_{14} \cdot x_{14}$$

$$p(\text{fraud} \mid x) = 1 / (1 + \exp(-z))$$

x_1 through x_{14} are the feature values for a transaction. The w values are fitted coefficients, b is the intercept, and $\exp$ is the exponential function. The sigmoid transformation makes the score lie between 0 and 1. The code does not print or save the Logistic Regression coefficients as a standalone fitted model.

The fitting procedure adjusts the coefficients so the training labels receive suitable probabilities. A linear combination can use every input together, but it is less flexible than arbitrary tree-based interactions. A probability describes the model's output, not a certainty about a transaction. The project does not perform probability calibration.

```
LogisticRegression(max_iter=1000, random_state=42)
```

Parameter	Meaning in this implementation
max_iter=1000	Upper limit of optimiser iterations; not 1,000 training records, trees, or guaranteed iterations.
random_state=42	Preserves the notebook setting. With the saved default lbfgs solver, this seed does not drive the stochastic shuffling used by some other solvers.
solver=lbfgs	Stored default numerical optimisation method.
penalty=l2; C=1.0	Stored defaults: coefficient regularisation with inverse regularisation strength C.
class_weight=None	No class weighting is specified. No scaling is applied in preprocessing.

API detail checked against the [scikit-learn 1.7 LogisticRegression reference](#); project parameter values are read from the saved artifact.

Actual project performance

Measure	Value	Meaning here
Precision	1.000000	All 205 transactions predicted as fraud were labelled fraud.
Recall	0.899123	205 of 228 fraud-labelled test transactions were detected; 23 were missed.
F1	0.946882	Harmonic mean of the fraud precision and recall.
ROC-AUC	0.991154	Strong ranking separation of fraud versus legitimate test transactions across thresholds.

Its test confusion matrix is $\begin{bmatrix} 2001 & 0 \\ 23 & 205 \end{bmatrix}$, with actual labels in rows and predicted labels in columns. Accuracy is $2,206 / 2,229 = 98.968147\%$. This result shows why perfect precision does not mean perfect detection: the model issued no false fraud alerts, yet failed to detect 23 fraud-labelled rows.

A useful viva distinction

`predict(X_test)` produces class labels for the confusion matrix, precision, recall, and F1.

`predict_proba(X_test)[:, 1]` produces class-1 probabilities for ROC-AUC. The default class decision is used; there is no custom threshold tuning in the source code. Increasing `max_iter` changes an optimisation limit, not the classification threshold.

08 Random Forest

Evidence: Notebook cells 43-44; `train.py`; saved `RandomForestClassifier` and its metadata; Phase 1 verification history.

Start with a decision tree

A decision tree applies a sequence of conditions to a row. At a split it might ask whether a numeric feature is below some learned value, then send the row down one branch. Continuing this process reaches a leaf. The training labels reaching a leaf determine its class proportions. This is an explanation of the model family; it is not a claim about a specific threshold in this saved forest.

During training, a tree searches for splits that make the resulting groups more homogeneous in their labels. A group containing only legitimate records is pure. A group mixing both labels is less pure. The saved forest uses the default gini criterion, a way of measuring class mixture.

```
Gini impurity = 1 - sum(p_class^2)
For two classes: 1 - p0^2 - p1^2
```

`p0` and `p1` are the class fractions among records at a node. A pure node has impurity zero. A node evenly divided between two classes has impurity 0.5. Splits are evaluated by how much they reduce weighted impurity. Those reductions also contribute to the feature-importance values discussed later.

Why use many trees?

A single tree can adapt strongly to peculiarities in its training sample. A Random Forest trains many trees with variation in their training samples and candidate features, then combines their outputs. This is ensemble learning: a collection of fitted models works together. Averaging can reduce sensitivity to any one tree's decisions, though it does not remove every source of error or sampling bias.

The saved defaults include `bootstrap=True`, which samples training rows with replacement for individual trees. A sampled row can occur more than once and another can be absent from a particular tree's sample. `max_features="sqrt"` restricts candidate features at splits. With 14 inputs, the nominal candidate count is the integer square-root setting, rather than always considering all 14. The actual model has 100 fitted trees.

```
RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)
```

Parameter	Detailed interpretation
<code>n_estimators=100</code>	Train 100 decision trees. This does not mean 100 features, rows, epochs, or probabilities.
<code>random_state=42</code>	Controls forest randomness such as bootstrap samples and feature selection so the same environment and inputs can reproduce the model.
<code>n_jobs=-1</code>	Use all available processors for supported parallel work. It changes execution parallelism, not the number of trees or a fraud threshold.
<code>max_depth=None</code>	No explicit maximum tree depth is imposed. Other stopping conditions still apply.
<code>min_samples_split=2; min_samples_leaf=1</code>	Stored defaults for the minimum records required to split and the minimum allowed leaf size.
<code>class_weight=None; oob_score=False</code>	No class weighting and no out-of-bag score calculation are requested.

How the forest produces a probability and label

A beginner can think of the trees as a committee. For this scikit-learn classifier, the precise mechanism averages the trees' class probabilities. A tree's probability is based on the training class fractions at the leaf where the transaction arrives. The forest chooses the class with the larger averaged probability. Thus "voting" is a helpful analogy, but the implementation is probability averaging rather than necessarily a count of hard 0/1 votes.

Forest fraud probability = $(p_1 + p_2 + \dots + p_{100}) / 100$

Here each p_t is a tree's estimated probability for fraud on the same transaction. The actual first workbook row returns `fraud_probability=0.97` and `isFraud=1` when passed to the saved predictor. That does not establish a calibrated 97% real-world chance; it reports this fitted model's score.

Aggregation and parallelism details checked against the [scikit-learn 1.7 RandomForestClassifier reference](#).

Why it can be useful for these inputs

Trees can combine transaction type, balances, amounts, and engineered differences through successive conditions. They can represent interactions and nonlinear boundaries without this project adding explicit interaction terms or scaling. These are capabilities of the model family, not evidence that any particular feature causes fraud. The unusually informative step feature remains an important qualification to the observed performance.

Actual project results

On 2,229 held-out transactions, the forest correctly classified all 2,001 legitimate rows and detected 227 of 228 fraud-labelled rows. It made zero false-positive predictions and one false-negative prediction. Precision is 1.000000, recall 0.995614, F1 0.997802, and ROC-AUC 0.999957. Accuracy is 99.955137%, commonly rounded to 99.96%.

The saved forest is the model fitted on the 8,913 training rows. `train.py` does not refit it on all 11,142 rows after evaluation. Its notebook business analysis and saved Phase 1 predictor therefore refer to the same train/test methodology.

The Windows permission issue during verification

The Phase 1 execution history records a Windows PermissionError, WinError 5, while joblib was creating worker infrastructure for `n_jobs=-1` inside the restricted execution sandbox. The traceback involved ThreadPool and Windows pipe setup. Training succeeded in a fresh process after an approved run outside that sandbox. The model configuration was not changed.

This is an execution-environment restriction, not a general rule that every Random Forest requires administrator rights or special permission on Windows. It does not indicate an error in the fraud data. The incident is evidenced by the earlier verification session, not stored as a log inside the project artifact.

09 Gradient Boosting

Evidence: Notebook cells 45-46; train.py; saved model_parameters and evaluations.

Concept: Gradient Boosting builds a sequence of small trees. Each new tree is fitted to a correction signal derived from the current model's loss. The combined model is updated gradually. For binary classification in this implementation, the loss is `log_loss`. The internal trees predict numeric corrections; their outputs combine to produce the classifier's final score and probability.

$$F_m(x) = F_{(m-1)}(x) + \text{learning_rate} * h_m(x)$$

F_m is the combined score after stage m . $F_{(m-1)}$ is the previous score. h_m is the new tree's correction. The learning rate scales how strongly that correction changes the model. For classification, the corrections are tied to the negative gradient of the loss, not simply to a list of rows previously assigned the wrong class.

An intuitive analogy

Imagine revising an answer through several rounds. The first draft is rough. Each revision focuses on what the current draft still gets wrong, rather than starting independently. Random Forest resembles multiple independent reviewers combining opinions; boosting resembles staged corrections to a shared answer. These analogies explain the difference without claiming that either model always performs better.

```
GradientBoostingClassifier(
    n_estimators=100, learning_rate=0.1,
    max_depth=3, random_state=42
)
```

Setting	Meaning
<code>n_estimators=100</code>	100 boosting stages. Binary classification uses one tree per stage here.
<code>learning_rate=0.1</code>	Scales each stage's contribution by 0.1. It is not a 10% fraud probability or a 10% train/test split.
<code>max_depth=3</code>	Limits each component tree's depth. It does not limit the whole ensemble to three stages.
<code>random_state=42</code>	Fixes the implementation's random choices, including feature-order effects while finding splits.
<code>subsample=1.0</code>	Stored default: each stage uses all training samples rather than a reduced stochastic subsample.

Setting	Meaning
n_iter_no_change=None	No early-stopping patience is configured; validation_fraction=0.1 is a default but does not establish an active validation split here.

Stage and parameter details checked against the [scikit-learn 1.7 GradientBoostingClassifier](#) reference.

Actual results and what they mean

Precision is 1.000000: all predicted frauds are fraud-labelled. Recall is 0.995614: it detects 227 of 228 fraud cases. F1 is 0.997802. ROC-AUC is 0.999678. The confusion-matrix counts match Random Forest: `[[2001, 0], [1, 227]]`. Matching counts do not, by themselves, prove every individual prediction or score is identical.

The probability ranking differs enough to produce a slightly smaller ROC-AUC than Random Forest, despite matching threshold-based metrics. This illustrates that ranking quality and one set of class decisions measure different aspects of performance. The fitted Gradient Boosting estimator is not included in the saved model bundle; its parameters, metrics, and classification report are included.

10 Model evaluation

Evidence: Notebook cells 41-46 and 61-63; artifact evaluations. Formulas below explain the existing metrics.

Positive means fraud

For every formula in this guide, class 1 is the positive class and class 0 is the negative class. “Positive” is a label convention, not a value judgement. A prediction is compared with the known test label to decide whether it was right or wrong.

Actual / Predicted	Predicted legitimate (0)	Predicted fraud (1)
Actual legitimate (0)	TN = 2,001	FP = 0
Actual fraud (1)	FN = 1	TP = 227

True Positive (TP)

Count of rows with actual=1 and predicted=1

227 fraud-labelled test transactions were correctly flagged. This is the detected-fraud count.

True Negative (TN)

Count of rows with actual=0 and predicted=0

2,001 legitimate test transactions were correctly left unflagged.

False Positive (FP)

Count of rows with actual=0 and predicted=1

0 legitimate transactions were incorrectly flagged as fraud. In a deployed workflow, such errors could create unnecessary review or disruption.

False Negative (FN)

Count of rows with actual=1 and predicted=0

1 fraud-labelled transaction was incorrectly classified as legitimate. This is missed fraud, which the notebook links to a potential missed amount.

$N = TP + TN + FP + FN = 2,229$ is the total test count. Actual fraud count is $TP + FN = 228$. Actual legitimate count is $TN + FP = 2,001$. Predicted fraud count is $TP + FP = 227$. The confusion matrix is an arrangement of these four counts; it is not one scalar score.

Accuracy: how many total decisions were correct?

$$\begin{aligned}\text{Accuracy} &= (TP + TN) / (TP + TN + FP + FN) \\ &= (227 + 2001) / 2229 \\ &= 0.9995513683266039 = 99.95513683266039\%\end{aligned}$$

The numerator counts correct decisions for both classes; the denominator counts all test decisions. Accuracy is easy to explain but can be dominated by the majority class. A hypothetical predictor that always returned legitimate would have $2,001 / 2,229 = 89.771198\%$ accuracy on this test split and zero fraud recall. That calculation is a teaching comparison, not a fourth model trained in the project.

train.py does not call accuracy_score separately. Accuracy is available inside the saved classification_report and can be derived from the confusion matrix, as above.

Precision: when we flag fraud, how often are we right?

$$\begin{aligned}\text{Precision} &= TP / (TP + FP) \\ &= 227 / (227 + 0) = 1.0\end{aligned}$$

The denominator includes only transactions predicted as fraud. Precision therefore addresses the trustworthiness of alerts. It matters when false alerts consume analyst attention or disrupt legitimate transactions. Here every one of the 227 fraud alerts matched a fraud label. It does not say the model found every fraud.

Recall: of all frauds, how many did we find?

$$\begin{aligned}\text{Recall} &= TP / (TP + FN) \\ &= 227 / (227 + 1) \\ &= 0.9956140350877193 = 99.56140350877193\%\end{aligned}$$

The denominator contains all actual fraud-labelled test rows, including the one missed. Recall is also called sensitivity or the true-positive rate. It matters when missed fraud is expensive. High recall here describes this split; it is not a guarantee about future cases.

F1: combine precision and recall

$$\begin{aligned}\text{F1} &= 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall}) \\ &= 2 * TP / (2 * TP + FP + FN) \\ &= 454 / 455 = 0.9978021978021978\end{aligned}$$

F1 is the harmonic mean, which penalises an imbalance between precision and recall more than an ordinary arithmetic mean does. If one of them is poor, the other cannot fully compensate. F1 is useful when both false alarms and missed fraud matter, but it contains no actual monetary cost and does not directly reward true negatives.

ROC: vary the decision threshold

True-positive rate (TPR) = $TP / (TP + FN)$

False-positive rate (FPR) = $FP / (FP + TN)$

A receiver operating characteristic curve plots TPR on the vertical axis against FPR on the horizontal axis as the score threshold changes. Lowering a fraud threshold can flag more rows, potentially increasing both detected fraud and false alarms. The Random Forest's current class decisions correspond to TPR=227/228 and FPR=0/2001. This single point is not the whole curve.

The notebook computes ROC-AUC from probabilities; it does not draw an ROC curve or tune a threshold. Explaining the curve here gives context for the existing metric, without claiming the project generated that additional chart.

ROC-AUC: area under the ranking curve

AUC = integral from FPR=0 to 1 of $TPR(FPR) \, d(FPR)$

Pairwise meaning = $P(\text{score}(\text{fraud}) > \text{score}(\text{legitimate}))$
 $+ 0.5 * P(\text{equal scores})$

The integral describes the area beneath the ROC curve. The pairwise interpretation asks whether a randomly chosen positive row gets a higher score than a randomly chosen negative row, with ties receiving half credit. An AUC of 1 means perfect ordering on the evaluated sample; an uninformative random ranking is around 0.5. AUC is not the proportion of correct class labels and is not the probability that an individual transaction is fraudulent.

Random Forest ROC-AUC is **0.9999572582129987**. The notebook rounds it to 1.0000 in its business narrative. The unrounded value is below 1, so it should not be described as mathematically perfect separation.

Metric definitions and score-versus-label usage checked against the [scikit-learn 1.7 model evaluation guide](#).

How to read classification_report

The class-1 row gives the fraud precision, recall, F1, and support=228. The class-0 row treats legitimate as the class of interest. Support is the number of actual rows of a class, not the number predicted as that class. Macro average gives each class equal weight; weighted average weights by class support. The project comparison table deliberately reports the class-1 metrics, whereas the full report contains both classes and overall accuracy.

Common mistakes to avoid

- Do not swap FP and FN: FP is a false fraud alarm; FN is missed fraud.
- Do not divide precision by all actual frauds; that denominator belongs to recall.
- Do not call ROC-AUC accuracy or interpret a rounded 1.0000 as exact perfection.
- Do not infer a business loss directly from an error count without looking up the transaction amounts.
- Do not claim high test performance proves real-world reliability beyond the supplied data.

11 Model comparison

Evidence: Notebook cell 54; saved artifact evaluations; Phase 1 training output.

Model	Precision	Recall	F1	ROC-AUC
Logistic Regression	1.000000	0.899123	0.946882	0.991154
Random Forest	1.000000	0.995614	0.997802	0.999957
Gradient Boosting	1.000000	0.995614	0.997802	0.999678

The table uses six decimal places for quick reading. Every value was checked against the saved artifact. The complete stored precision is shown below so the reproduction result is auditable.

Logistic Regression

Precision: 1.0000000000000000
 Recall: 0.899122807017544
 F1: 0.946882217090069
 ROC-AUC: 0.991153546034001

Random Forest

Precision: 1.0000000000000000
 Recall: 0.995614035087719
 F1: 0.997802197802198
 ROC-AUC: 0.999957258212999

Gradient Boosting

Precision: 1.0000000000000000
 Recall: 0.995614035087719
 F1: 0.997802197802198
 ROC-AUC: 0.999677792682606

What the numerical differences show

All three models have precision 1.0 and zero false positives on this test set. Logistic Regression detects 205 fraud cases; each tree ensemble detects 227. That is 22 additional detected fraud cases in the aggregate counts, or a recall difference of about 9.6491 percentage points. Its 23 false negatives explain why Logistic Regression has the lower F1.

Random Forest and Gradient Boosting have equal precision, recall, F1, and confusion-matrix counts. Their ROC-AUC values differ by approximately 0.000279466, with Random Forest numerically higher. That small difference measures ordering of scores, not 0.0279% more detected transactions at the current class decision.

The notebook uses Random Forest for business reporting, and Phase 1 saves it for prediction. There is no statistical significance test, runtime benchmark, hyperparameter search, or universal model-ranking claim. No evidence in these files supports saying that Random Forest must outperform Gradient Boosting on every future dataset.

12 Feature importance

Evidence: Notebook cells 50-53; saved Random Forest feature_importances_.

Feature importance describes how much a feature contributed to the fitted trees' impurity reductions in this dataset. Larger values mean the tree construction relied more on that feature under this measure. The values are normalised within a model, so their total is approximately 1. An importance of 0.446 is a share of this importance measure, not 44.6% accuracy and not a fraud probability.

Random Forest: full-precision evidence

Feature	Importance
step	0.445970908064859
orig_balance_change	0.141126442410986
orig_balance_error	0.123547102931907
newbalanceOrig	0.070307242000189
amount	0.065764058036085
type_TRANSFER	0.030432061446091
newbalanceDest	0.022624425967215
oldbalanceDest	0.020765963356331
type_PAYMENT	0.018977798672023
type_CASH_OUT	0.016189962584904
dest_balance_error	0.015107081812188
oldbalanceOrg	0.014915146954388
dest_balance_change	0.012774171263266
type_DEBIT	0.001497634499567

step is the largest contributor, approximately 44.60%. orig_balance_change contributes approximately 14.11%; orig_balance_error approximately 12.35%; newbalanceOrig approximately 7.03%; and amount approximately 6.58%. Other balances, balance errors, and transaction indicators account for the remaining importance.

Gradient Boosting: saved display precision only

Feature (notebook order)	Notebook displayed importance
step	0.93
newbalanceOrig	0.05
orig_balance_change	0.01
newbalanceDest	0.00
oldbalanceDest	0.00
oldbalanceOrg	0.00
orig_balance_error	0.00
amount	0.00
dest_balance_error	0.00
type_TRANSFER	0.00
type_CASH_OUT	0.00
dest_balance_change	0.00
type_DEBIT	0.00
type_PAYMENT	0.00

The notebook displays these Gradient Boosting values at two decimal places because of its global float-format setting. The saved bundle contains no fitted Gradient Boosting model and no full-precision Gradient Boosting importance array. Therefore exact unrounded values cannot be recovered from the supplied saved files. In particular, 0.00 here may mean a small nonzero value rounded to zero. The displayed values need not sum to exactly 1 after rounding.

Interpretation and the unresolved step question

The step feature separates much of the sample in time: legitimate records occur at steps 1-7, whereas fraud extends to step 95. A tree can exploit this separation. High step importance is consistent with that sample pattern and with the notebook's concern about sampling or temporal leakage. It does not establish that waiting an extra hour causes fraud.

Engineered changes and errors provide information about the relationship between amount and recorded balances. newbalanceOrig describes the remaining origin balance; amount describes transaction size. Because these variables are related, importance can be shared or redistributed among them. A low importance is not proof a variable is useless in all other models. No causal analysis, permutation importance, or SHAP analysis was performed in the project.

13 Financial impact analysis

Evidence: Notebook cells 55-59; workbook amount column; saved Random Forest test predictions verified against the notebook split.

Measure	Actual value	Scope
Total fraudulent transactions	1,142	Entire dataset
Total fraud amount	1,361,982,240.43	Sum of amounts for all 1,142 fraud-labelled rows
Missed fraudulent transactions	1	Random Forest test set only
Potential missed-fraud loss	416,001.33	Amount of the missed test transaction
False-positive transactions	0	Random Forest test set only
False-positive amount	0.00	Sum of amounts for false-positive test rows

Total fraud amount

```
fraud_transactions = df[df["isFraud"] == 1]
total_fraud_transactions = len(fraud_transactions)
total_fraud_loss = fraud_transactions["amount"].sum()
```

The Boolean filter keeps every fraud-labelled row from the full dataset. `len` counts those rows. `sum` adds their amount values. Despite the variable name `total_fraud_loss`, the available evidence is a sum of recorded transaction amounts, not a reconciled statement of realised unrecovered losses. No recovery or reimbursement fields are present.

Missed fraud: align the test predictions to the source rows

```
rf_false_negative = (y_test == 1) & (rf_model.predict(X_test) == 0)
missed_fraud_loss = df.loc[
    y_test.index[rf_false_negative], "amount"
].sum()
```

The first condition means actual fraud; the second means predicted legitimate. The ampersand combines the conditions element by element. Parentheses keep each comparison separate. `y_test` retains original DataFrame indexes after splitting, so `y_test.index[mask]` identifies the source rows. `loc` retrieves amount values by those original labels. This prevents confusing shuffled test positions with original row positions.

Exactly one row meets the condition, with amount 416,001.33. The notebook calls this potential missed-fraud loss. It assumes the missed transaction amount is the relevant exposure; it does not implement a separate cost model.

False positives

```
rf_false_positive = (y_test == 0) & (rf_model.predict(X_test) == 1)
false_positive_amount = df.loc[
    y_test.index[rf_false_positive], "amount"
].sum()
```

This mask selects legitimate records wrongly flagged as fraud. There are none, so the sum is 0.00. A transaction amount associated with a false alert would not automatically equal the cost of that alert: analyst time or customer disruption would need additional data. The project does not estimate those separate costs.

Business interpretation and currency

A high recall can still leave a material exposure if the missed transaction is large. Reporting both the number of missed cases and their amounts makes this visible. Do not compare the full-dataset total with test-set missed amounts as if both were calculated on the same population, and do not infer a deployment-wide loss rate from this single split.

The notebook prints the rupee symbol. No currency metadata in the supplied workbook establishes that the amounts are denominated in INR. The guide therefore preserves the numbers and identifies the notebook's currency presentation as an unverified interpretation. `train.py` stores evaluation metrics, but financial-impact sums remain notebook-only reporting logic.

14 Business questions

Evidence: Notebook cells 61-63; Random Forest saved evaluation.

1. What is the model's precision and accuracy in detecting fraudulent transactions?

Actual answer: Random Forest precision is 1.0000 and accuracy is approximately 99.96%. Recall is approximately 99.56%, F1 is 0.9978, and the exact ROC-AUC is 0.9999572582129987.

How obtained: Precision divides 227 correct fraud alerts by 227 total fraud alerts. Accuracy divides 2,228 correct decisions by 2,229 test decisions. These use the saved confusion matrix, not a training score.

Meaning for a stakeholder: In this evaluation, alerts were highly precise and overall classification was strong. Precision alone does not guarantee all fraud was caught, so include recall and the missed-case count when presenting this answer.

2. How reliable is the model in classifying transactions as legitimate or fraudulent?

Actual answer: It detected 227 of 228 fraud-labelled test transactions, missed one, and produced zero false positives among 2,001 legitimate test transactions. The notebook describes this as very reliable on the test dataset.

How obtained: Read the four cells of the confusion matrix and the class-1 report. These demonstrate reliability on this test split. There is no external or live-production evaluation in the files.

Meaning for a stakeholder: The observed error count is low, but the scope must remain explicit. The unresolved time-step pattern may contribute to the unusually high results, and the tested sample is the supplied dataset.

3. What are the potential losses due to model errors?

Actual answer: The one missed fraud has recorded amount 416,001.33. False-positive transaction count is zero and associated transaction amount is 0.00. The notebook presents those values with a rupee symbol.

How obtained: Select false-negative and false-positive rows using actual labels and predictions, preserve their original indexes, then sum amount. This is an amount-based exposure calculation, not a full profit-and-loss model.

Meaning for a stakeholder: Even one missed case may matter financially. A deployment decision would require more than a high percentage score; this project's concrete evidence is the held-out error counts and associated amounts, with currency provenance unresolved.

15 Python project structure

Evidence: Actual directory inventory and all seven primary source files.

```
Fraud Detection Capstone Project/
|-- Fraud_Detection_Capstone_Starter (4).ipynb
|-- Fraud_Analysis_Dataset.xlsx
|-- requirements.txt
|-- src/
|   |-- preprocessing.py
|   |-- train.py
|   |-- predict.py
|-- models/
|   |-- random_forest.joblib
```

This tree shows the machine-learning implementation before the guide output and temporary authoring files. No Streamlit module or configuration is present. There is no separate schema JSON because preprocessing metadata is bundled inside the joblib file.

File	Responsibility
Original notebook	Primary analytical narrative: EDA, plots, experiments, interpretation, and business questions.
Excel dataset	Unchanged raw transaction records and labels.
src/preprocessing.py	Portable paths, deterministic feature engineering, training encoding, and inference feature alignment.
src/train.py	Split, fit three estimators, evaluate, compare against notebook metrics, and save the Random Forest bundle.
src/predict.py	Load saved artifact and classify one supplied transaction without training.
models/random_forest.joblib	Binary serialised bundle containing fitted forest and accompanying metadata.
requirements.txt	Exact package versions required by the Phase 1 implementation.

Separating responsibilities allows the same preprocessing to serve training and prediction. It also lets a future interface call a prediction function without copying notebook cells or rerunning model fitting. The notebook remains intact as the analytical reference; the scripts provide a reproducible execution path.

How to run the module

```
python -B -m src.train
```

Run this from the project root with the required environment active. `-m` executes `src.train` as a module so its relative import from `.preprocessing` works. `-B` disables bytecode-cache writing; it does not change the

training algorithm. The src directory functions as a namespace package without an `__init__.py`. Running `python src/train.py` directly is not the documented invocation for its relative imports.

The verified interpreter is the existing Anaconda Python 3.13.9 environment. The separate system Python 3.12.10 inspected during Phase 1 did not have the required machine-learning packages installed. Choosing the correct interpreter is part of reproducing the run.

16 preprocessing.py: code walkthrough

Evidence: Complete current `src/preprocessing.py`. Line numbers below refer to this inspected file.

This module has four reusable functions and three groups of constants. It contains no estimator fitting, no evaluation metrics, and no file saving. The code excerpts collectively cover every nonblank part of the file. Long display lines are visually wrapped; the source file itself is unchanged.

Imports and portable paths (lines 1-9)

```
1  """Notebook-equivalent feature preparation for training and prediction."""
2
3  from pathlib import Path
4
5  import pandas as pd
6
7  PROJECT_ROOT = Path(__file__).resolve().parents[1]
8  DATASET_PATH = PROJECT_ROOT / "Fraud_Analysis_Dataset.xlsx"
9  MODEL_PATH = PROJECT_ROOT / "models" / "random_forest.joblib"
```

The first triple-quoted string documents the module. `Path` comes from Python's standard `pathlib` library. `pandas` is imported under the conventional alias `pd`. `__file__` is this module's path; `resolve()` makes it absolute; `parents[1]` selects the project root one level above `src`. The `/` operator joins path components. `DATASET_PATH` points to the existing workbook and `MODEL_PATH` to the saved bundle.

If removed or changed: Using a bare filename would make loading depend on the caller's working directory. Moving the source file to a different directory depth without updating `parents[1]` would change the resolved root. Removing `pandas` would make the table operations unavailable.

Raw and engineered feature lists (lines 10-17)

```
10 RAW_FEATURES = [
11     "step", "amount", "oldbalanceOrig", "newbalanceOrig",
12     "oldbalanceDest", "newbalanceDest", "type",
13 ]
14 FEATURES = RAW_FEATURES[:-1] + [
15     "orig_balance_change", "dest_balance_change",
16     "orig_balance_error", "dest_balance_error", "type",
17 ]
```

`RAW_FEATURES` records the seven required transaction fields. `RAW_FEATURES[:-1]` takes all entries except the final type entry. Adding the four engineered names and type reconstructs the notebook's exact 11-column pre-encoding order. After type becomes four dummy columns, `X` has 14 columns.

If removed or changed: Changing the ordering or names could break compatibility with the trained model. Removing `step` would change the methodology. Including `isFraud` would leak the answer into the inputs; neither identifier is selected here.

load_dataset() (lines 20-22)

```
20 def load_dataset(path=DATASET_PATH):
21     """Resolve the default dataset relative to this project, not the working
    directory."""
22     return pd.read_excel(path)
```

The function accepts an optional path, defaulting to the project-relative DATASET_PATH. `pd.read_excel` reads the workbook's first worksheet and returns a DataFrame. In this workbook the first and only sheet is in. The return value is the in-memory table; reading does not overwrite Excel.

If removed or changed: Removing the return would cause callers to receive `None` instead of a DataFrame. Changing the default path could select a different dataset. There is no custom sheet selection or data cleaning hidden in this function.

Input validation and copying (lines 25-30)

```
25 def engineer_features(transactions):
26     """Copy inputs and apply the four original notebook formulas."""
27     missing = set(RAW_FEATURES) - set(transactions.columns)
28     if missing:
29         raise ValueError(f"Missing transaction fields: {sorted(missing)}")
30     frame = transactions.copy()
```

`transactions` is the incoming DataFrame. A set difference finds required raw column names not present in `transactions.columns`. An empty set is false in an if statement. If fields are missing, `ValueError` reports their sorted names; otherwise `copy()` creates a working table so adding derived columns does not mutate the caller's original DataFrame.

If removed or changed: Without the required-column check, an arithmetic operation would later fail with a less helpful `KeyError`. Without copying, feature assignment could alter a shared DataFrame. This check validates column presence, not all numeric ranges, missing numeric values, or transaction semantics.

Four formulas and explicit selection (lines 31-35)

```
31     frame["orig_balance_change"] = frame["oldbalanceOrig"] - frame["newbalanceOrig"]
32     frame["dest_balance_change"] = frame["newbalanceDest"] - frame["oldbalanceDest"]
33     frame["orig_balance_error"] = frame["amount"] - frame["orig_balance_change"]
34     frame["dest_balance_error"] = frame["amount"] - frame["dest_balance_change"]
35     return frame[FEATURES].copy()
```

Each bracket assignment creates or replaces one column in the working copy. Origin change is before minus after; destination change is after minus before. Each error is amount minus its corresponding change. `frame[FEATURES]` selects only the desired 11 pre-encoding inputs, preserving their list order. A second copy makes that selected table explicit and separate.

If removed or changed: Reversing subtraction changes feature meaning and trained-model compatibility. Omitting a derived column would alter the expected 14-feature matrix. Extra incoming columns, including account IDs or a supplied label, are discarded by the explicit selection.

Training matrix and labels (lines 38-42)

```
38 def prepare_training_data(frame):
39     """Encode before splitting, exactly as in the notebook; return X, y, schema."""
40     features = engineer_features(frame)
41     X = pd.get_dummies(features, columns=["type"], drop_first=True)
42     y = frame["isFraud"].copy()
```

engineer_features produces the selected pre-encoding table. get_dummies replaces type with four Boolean indicators and drops the first category. y is a copy of isFraud from the original frame. X and y retain the original row indexes. They are constructed before the split, exactly as in the notebook.

If removed or changed: If y came from an unrelated or differently ordered table, labels would no longer match transactions. Removing drop_first changes the schema. The function does not perform splitting, scaling, balancing, or fitting.

The schema dictionary (lines 43-51)

```

43     schema = {
44         "type_categories": sorted(features["type"].dropna().unique().tolist()),
45         "feature_columns": X.columns.tolist(),
46         "feature_dtypes": {name: str(dtype) for name, dtype in X.dtypes.items()},
47         "raw_features": RAW_FEATURES.copy(),
48         "encoding": "pandas.get_dummies",
49         "drop_first": True,
50     }
51     return X, y, schema

```

schema is a Python dictionary of preprocessing metadata. dropna().unique().tolist() obtains distinct non-null type labels and sorted() fixes their order. feature_columns stores the exact encoded column order. The dictionary comprehension over X.dtypes.items() stores each dtype as text. raw_features is a copied list of required inputs; encoding is a descriptive string; drop_first stores the encoding option. return X, y, schema returns three objects as a tuple.

If removed or changed: Without categories, a single-row prediction cannot reliably recreate training dummy columns. Without feature order, values might reach the wrong model positions. raw_features and encoding are descriptive metadata in this implementation; they are saved but are not dynamically interpreted to rebuild arbitrary formulas.

Prediction categories and validation (lines 54-62)

```

54 def prepare_prediction_data(transactions, schema):
55     """Use training categories so a single row gets the same dummy columns."""
56     features = engineer_features(transactions)
57     valid = features["type"].isin(schema["type_categories"])
58     if not valid.all():
59         raise ValueError("Transaction type must be one of " +
60                           str(schema["type_categories"]))
61     features["type"] = pd.Categorical(
62         features["type"], categories=schema["type_categories"]
63     )

```

Prediction uses the same engineer_features function. isin checks each type against the saved training categories. valid.all() requires every row to pass; otherwise a ValueError lists the allowed categories. pd.Categorical declares all training categories even when the incoming batch contains only one type. This is what makes single-transaction encoding reliable.

If removed or changed: If unknown categories were silently accepted, they could become all-zero indicators and be confused with the CASH_IN baseline. If Categorical were omitted, get_dummies could build columns from only the categories in the new row or batch.

Encoding, alignment, and dtype restoration (lines 63-69)

```

63     X = pd.get_dummies(features, columns=["type"], drop_first=schema["drop_first"])
64     # Preserve numeric input precision; dummy dtypes match the saved training schema.
65     X = X.reindex(columns=schema["feature_columns"])
66     for name in X.columns:
67         if name.startswith("type_"):
68             X[name] = X[name].astype(schema["feature_dtypes"][name])
69     return X

```

`get_dummies` uses the saved `drop_first` flag. `reindex(columns=...)` arranges columns to match the training schema. The for loop checks each column name; columns beginning `type_` are cast to their saved Boolean dtype. Numeric inputs are not force-cast to the recorded numeric dtypes, preserving incoming numeric precision. The function returns `X` ready for the model.

If removed or changed: Omitting alignment could violate the estimator's expected feature names or order. Removing Boolean casting may alter schema consistency. The current numeric handling does not guarantee that invalid strings or missing numeric values are rejected with a custom message; it relies on arithmetic and estimator validation for those cases.

What the module guarantees and what it does not

It guarantees the implemented four formulas, required raw column names, allowed prediction categories, and saved encoded feature ordering. It does not encapsulate the formulas inside the joblib artifact; future predictions still require this compatible Python module. It also does not validate all business rules, such as whether a balance movement is plausible for a transaction type. Those are implementation boundaries, not extra transformations performed behind the scenes.

17 train.py: code walkthrough

Evidence: Complete current `src/train.py`. The artifact was produced by this implementation.

Imports and module entry convention (lines 1-16)

```

1  """Run with python -B -m src.train from the project root."""
2
3  import platform
4  from importlib.metadata import version
5
6  import joblib
7  import pandas as pd
8  from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
9  from sklearn.linear_model import LogisticRegression
10 from sklearn.metrics import (
11     classification_report, confusion_matrix, f1_score,
12     precision_score, recall_score, roc_auc_score,
13 )
14 from sklearn.model_selection import train_test_split
15
16 from .preprocessing import MODEL_PATH, load_dataset, prepare_training_data

```

The docstring gives the supported module command. `platform` supplies the Python version; `importlib.metadata.version` reads installed distribution versions. `joblib` saves the bundle and `pandas` creates the comparison table. The `sklearn` imports provide the three estimator classes, six evaluation/reporting functions, and the split function. The leading dot in `.preprocessing` means import from the same `src` package.

If removed or changed: Missing an imported package prevents execution. Running the file directly instead of as a module can break the relative import. Matplotlib, seaborn, and Streamlit are not imported because this script prints reports rather than building plots or a UI.

Notebook reference metrics (lines 18-23)

```
18 NOTEBOOK_METRICS = {
19     "Logistic Regression": [1.0, 0.8991228070175439, 0.9468822170900693,
20         0.9911535460340006],
21     "Random Forest": [1.0, 0.9956140350877193, 0.9978021978021978,
22         0.9999572582129987],
23     "Gradient Boosting": [1.0, 0.9956140350877193, 0.9978021978021978,
24         0.999677792682606],
25 }
26 METRIC_NAMES = ["Precision", "Recall", "F1", "ROC-AUC"]
```

NOTEBOOK_METRICS maps each model name to four known results in the order defined by METRIC_NAMES: Precision, Recall, F1, ROC-AUC. These constants are comparison targets, not fitted parameters and not inputs to training. They match the notebook's saved outputs.

If removed or changed: Changing them could hide a reproduction discrepancy. Changing metric order without changing both lists would compare unrelated quantities. The constants do not force model predictions to match; training must independently produce the observed scores.

Loading, preprocessing, and splitting (lines 26-30)

```
26 def train():
27     X, y, schema = prepare_training_data(load_dataset())
28     X_train, X_test, y_train, y_test = train_test_split(
29         X, y, test_size=0.20, random_state=42, stratify=y
30     )
```

train() orchestrates the pipeline. The inner load_dataset() returns the raw workbook table. prepare_training_data transforms it and returns X, y, and schema, unpacked into three variables. train_test_split then returns training inputs, test inputs, training labels, and test labels in that order. test_size=0.20, random_state=42, and stratify=y preserve the notebook methodology.

If removed or changed: Changing data order, split seed, or stratification can change the test examples and metrics. Running preprocessing after splitting would alter the notebook's original encoding workflow, even if this dataset happened to produce equivalent columns.

Constructing estimators and result containers (lines 31-39)

```
31 models = {
32     "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
33     "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42,
34         n_jobs=-1),
35     "Gradient Boosting": GradientBoostingClassifier(
36         n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42
37     ),
38 }
39 print(f"Features: {X.shape[1]} | Training: {len(X_train)} | Testing:
40 {len(X_test)}")
41 rows, evaluations, discrepancies = [], {}, []
```

models is an insertion-ordered dictionary whose keys are display names and values are unfitted estimator instances. The constructors specify exactly the notebook settings. The print statement shows feature and partition sizes. rows is an empty list for table rows, evaluations is a dictionary for detailed outputs, and discrepancies is a list of reproduction failures.

If removed or changed: Changing estimator settings changes the experiment. Removing a container without replacing its later use would cause errors. Instantiating an estimator does not train it; fitting happens in the next block.

Fitting, predicting, and scoring (lines 40-47)

```

40     for name, model in models.items():
41         model.fit(X_train, y_train)
42         predicted = model.predict(X_test)
43         probability = model.predict_proba(X_test)[: , 1]
44         scores = [
45             precision_score(y_test, predicted), recall_score(y_test, predicted),
46             f1_score(y_test, predicted), roc_auc_score(y_test, probability),
47         ]

```

The loop visits each model. `fit(X_train, y_train)` learns its parameters using only training rows. `predict(X_test)` produces class labels. `predict_proba(X_test)` returns one probability column per class; `[:, 1]` selects the class-1 column for this known binary target. The score list calculates precision, recall, F1 from labels, and ROC-AUC from probabilities.

If removed or changed: Fitting on `X_test` would invalidate the holdout evaluation. Computing ROC-AUC from hard labels would discard ranking information. Removing `fit` would leave an unfitted estimator that cannot predict. No threshold search, model tuning, or scaling takes place in the loop.

Collecting reports and printing matrices (lines 48-56)

```

48         rows.append({"Model": name, **dict(zip(METRIC_NAMES, scores))})
49         evaluations[name] = {
50             "metrics": dict(zip(METRIC_NAMES, scores)),
51             "confusion_matrix": confusion_matrix(y_test, predicted).tolist(),
52             "classification_report": classification_report(y_test, predicted,
output_dict=True),
53         }
54         print(f"\n{name}\n{classification_report(y_test, predicted)}")
55         print("Confusion matrix (actual rows, predicted columns; labels 0, 1):")
56         print(confusion_matrix(y_test, predicted))

```

`zip` pairs each metric name with its score. `dict` converts those pairs to a mapping, and `**` merges the mapping into a row that also includes the model name. `evaluations[name]` stores metrics, a list representation of the confusion matrix, and a dictionary-form classification report. Separate print calls display the readable report and matrix. `tolist()` converts a NumPy matrix to ordinary nested lists for convenient metadata storage.

If removed or changed: Removing the evaluation dictionary would lose the detailed reports inside the bundle. Removing print calls affects console visibility but not fitted predictions. Misreading matrix orientation would reverse business interpretations of false positives and false negatives.

Reproduction guard and comparison table (lines 57-68)

```

57         for metric, actual, expected in zip(METRIC_NAMES, scores,
NOTEBOOK_METRICS[name]):
58             if abs(actual - expected) > 1e-6:
59                 discrepancies.append(f"{name} {metric}: {actual:.15f} vs notebook
{expected:.15f}")
60
61         comparison = pd.DataFrame(rows)
62         print("\nModel comparison")
63         print(comparison.to_string(index=False, float_format=lambda value:
f"{value:.15f}"))
64         if discrepancies:
65             raise RuntimeError(
66                 "Notebook reproduction differs by more than 0.000001; model was not
saved.\n"
67                 + "\n".join(discrepancies)
68             )

```

The inner zip aligns metric labels, freshly calculated scores, and notebook reference scores. `abs(actual - expected)` calculates absolute difference. If it exceeds `1e-6`, a precise discrepancy message is stored. After all models, pandas builds a comparison table and `to_string` prints 15 decimal places without the row index. A nonempty discrepancy list triggers `RuntimeError` before saving.

If removed or changed: Removing this guard could silently replace the artifact with a result that does not reproduce the notebook. The guard is a reproducibility check for this fixed project, not a model-selection optimisation. If a previous artifact already exists and a later run fails this check, that older file remains; the code does not delete it.

The saved artifact dictionary (lines 70-81)

```

70     artifact = {
71         "model": models["Random Forest"],
72         "preprocessing": schema,
73         "evaluations": evaluations,
74         "model_parameters": {name: model.get_params() for name, model in
models.items()}},
75     "split": {"test_size": 0.20, "random_state": 42, "stratify": "isFraud"},
76     "versions": {
77         "python": platform.python_version(),
78         **{name: version(name) for name in
79            ["pandas", "numpy", "scipy", "scikit-learn", "openpyxl", "joblib"]},
80     },
81 }

```

model selects the fitted Random Forest object. preprocessing stores the schema; evaluations stores all three models' reports. `get_params()` records complete constructor settings, including defaults, for every estimator. split records the declared split settings. versions combines the Python version with installed versions of pandas, NumPy, SciPy, scikit-learn, openpyxl, and joblib using a dictionary comprehension.

If removed or changed: Saving only the estimator would omit category and column alignment metadata. Saving only metrics would not allow inference. The artifact does not contain fitted Logistic Regression or Gradient Boosting objects, split row indexes, raw data, or a serialised copy of the preprocessing functions.

Saving, returning, and the main guard (lines 82-89)

```

82     MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)
83     joblib.dump(artifact, MODEL_PATH)
84     print(f"\nNotebook metrics reproduced within 0.000001. Saved: {MODEL_PATH}")
85     return comparison
86
87
88 if __name__ == "__main__":
89     train()

```

MODEL_PATH.parent is the models directory. mkdir(parents=True, exist_ok=True) creates it if absent and accepts an existing directory. joblib.dump serialises the artifact to the configured file. The success message reports that metrics passed the tolerance check. return comparison makes the DataFrame available to callers. The __name__ guard invokes train() only when the module is executed as the main program.

If removed or changed: Calling train() again would train all three models and overwrite the same model artifact if the reproduction check passes. Importing the module alone does not train because of the guard. Prediction uses a separate module and never calls this entry point.

A complete trace of the major variables

Name	Role
schema	The recipe metadata needed to align future input columns.
name / model	Current display label and estimator inside the loop.
predicted / probability	Hard class outputs and fraud scores on X_test.
scores / rows	One model's numeric metrics and the accumulated comparison records.
evaluations	Detailed per-model metrics, matrices, and classification reports.
discrepancies	Any metric differences exceeding absolute tolerance 0.000001.
comparison / artifact	Human-readable result table and the bundle written for inference.

18 predict.py: code walkthrough

Evidence: Complete current src/predict.py; verified saved-model prediction.

Inference imports (lines 1-6)

```
1 """Single-transaction inference using the saved Random Forest; never retrains."""
2
3 import joblib
4 import pandas as pd
5
6 from .preprocessing import MODEL_PATH, prepare_prediction_data
```

The module docstring states the inference-only purpose. joblib loads the saved bundle. pandas builds the one-row table expected by preprocessing. The relative import obtains the default model path and the shared prediction transformation function.

If removed or changed: Duplicating feature formulas here could make them diverge from training. Importing train() and fitting inside inference would increase latency and could change the model between requests; this implementation does neither.

The prediction function contract (lines 9-15)

```
9 def predict_transaction(transaction, model_path=MODEL_PATH):
10     """Accept a mapping of the seven raw features; return class and fraud probability.
11
12     Required: step, type, amount, oldbalanceOrig, newbalanceOrig,
13     oldbalanceDest, newbalanceDest. Account IDs and isFraud are not needed.
14     Only load model artifacts produced by this project or another trusted source.
15     """
```

predict_transaction takes a mapping, normally a Python dictionary, containing the seven required raw fields. model_path is optional and defaults to the project's artifact. The docstring lists required values and notes that account identifiers and isFraud are unnecessary. It also states that only trusted joblib artifacts should be loaded.

If removed or changed: Missing required fields trigger preprocessing validation. Passing a different artifact requires compatibility with this bundle structure and schema. Joblib uses Python object deserialisation, so loading an untrusted artifact is not the same as reading a harmless plain-text file.

Load, transform, and identify the fraud column (lines 16-19)

```
16 artifact = joblib.load(model_path)
17 X = prepare_prediction_data(pd.DataFrame([transaction]),
18     artifact["preprocessing"])
19 model = artifact["model"]
20 fraud_index = list(model.classes_).index(1)
```

joblib.load restores the dictionary. pd.DataFrame([transaction]) uses a one-element list so one dictionary becomes one row. prepare_prediction_data engineers the four features, validates type, encodes with saved categories, and aligns columns. artifact["model"] retrieves the fitted forest. model.classes_ stores class labels; converting it to a list and finding 1 determines the fraud probability column explicitly.

If removed or changed: Ignoring the saved schema could put values into wrong columns or omit category indicators. Assuming every future artifact uses the same class order is less robust than looking up label 1. No fit call is present. The bundle is loaded on every function call; no caching is implemented.

Returned label and probability (lines 20-23)

```

20     return {
21         "isFraud": int(model.predict(X)[0]),
22         "fraud_probability": float(model.predict_proba(X)[0, fraud_index]),
23     }

```

`predict(X)` returns an array with one class label; `[0]` selects that row and `int` converts it to a normal Python integer. `predict_proba(X)` returns a one-row, two-column array; `[0, fraud_index]` selects that transaction's fraud score. `float` converts it to a standard Python floating-point number. The return value is a dictionary with `isFraud` and `fraud_probability`.

If removed or changed: Returning only the label would discard the score; returning only the probability would require a caller to define a class decision. The existing function delegates classification to `model.predict` rather than applying a custom business threshold.

Example call using the actual first workbook row

```

from src.predict import predict_transaction

transaction = {
    "step": 1, "type": "TRANSFER", "amount": 181.0,
    "oldbalanceOrg": 181.0, "newbalanceOrig": 0.0,
    "oldbalanceDest": 0.0, "newbalanceDest": 0.0,
}
result = predict_transaction(transaction)
# Verified returned result:
# {"isFraud": 1, "fraud_probability": 0.97}

```

This call demonstrates the saved inference path. Because the example is drawn from the supplied workbook, it is not an independent external test or evidence of live deployment performance. The code shown is a usage example, not a newly created application file.

19 Joblib and model saving

Evidence: `models/random_forest.joblib`; `train.py` artifact construction; `predict.py` model loading.

A `.joblib` file is a binary serialisation of Python objects. Serialisation stores an object's state so a later Python process can reconstruct it. A fitted forest contains learned tree structures and values; saving it avoids rebuilding those structures for every prediction. The extension alone does not tell you what keys the file contains, so this artifact was loaded and inspected.

Key	Stored content	What it supports
model	Fitted RandomForestClassifier	Class labels and probabilities for new rows.
preprocessing	Categories, feature order, dtypes, raw fields, encoding name, drop_first	Compatible preparation of inference inputs.
evaluations	All three models' scores, confusion matrices, and reports	Audit of Phase 1 evaluation.
model_parameters	<code>get_params()</code> dictionaries for all three models	Explicit settings and inherited defaults.
split	<code>test_size</code> , <code>random_state</code> , stratify label	Description of the split methodology.
versions	Python and six package versions	Environment identification for reproducibility.

Saving and loading are different from fitting

```

joblib.dump(artifact, MODEL_PATH) # training: persist the fitted state
artifact = joblib.load(MODEL_PATH) # prediction: restore the fitted state
model = artifact["model"]
model.predict(X)                  # inference: apply learned state

```

Fitting learns from labelled examples; saving writes the fitted state; loading reconstructs it; prediction applies it. An application should not unexpectedly retrain each time a user opens a page or submits a row. Reusing the saved state makes results stable for a fixed model version and avoids repeated training cost.

The model bundle does not make the project self-contained independently of code and dependencies. It stores preprocessing metadata, but the four feature formulas remain in `preprocessing.py`. It also depends on compatible installed Python packages. The only fitted estimator in the bundle is Random Forest. No neural-network weights, Streamlit app, or alternative dataset is hidden in the artifact.

Trust and version compatibility

The `predict.py` docstring explicitly limits loading to trusted artifacts. Python-object deserialisation can execute code, so an arbitrary uploaded `.joblib` file is not a safe substitute for this project's known model. The saved versions identify the environment that produced the artifact; they are not automatically enforced by `predict.py`, and compatibility across arbitrary future versions is not established here.

20 requirements.txt

Evidence: Actual six-line requirements.txt and artifact versions.

```
pandas==2.3.3
scikit-learn==1.7.2
openpyxl==3.1.5
joblib==1.5.2
numpy==2.3.5
scipy==1.16.3
```

Package	Why this implementation needs it
pandas 2.3.3	DataFrames, Excel loading, feature arithmetic, dummy encoding, schema metadata, and result tables.
scikit-learn 1.7.2	The three estimators, train_test_split, prediction interfaces, evaluation metrics, and reports. Imported as sklearn.
openpyxl 3.1.5	The installed engine used by pandas to read the .xlsx workbook. It is not called directly in the source.
joblib 1.5.2	Explicit artifact saving/loading and parallelism support used within scikit-learn.
numpy 2.3.5	Required numerical-array dependency underlying pandas and scikit-learn. Not imported directly by these three modules.
scipy 1.16.3	Required scientific-computing dependency of scikit-learn, including numerical optimisation infrastructure. Not directly imported in project modules.

The double equals pins an exact version. NumPy and SciPy are explicitly pinned even though the project does not directly import them, because they are necessary numerical dependencies and their versions are part of the verified environment. The file is not an exhaustive lockfile of every transitive dependency or operating-system component.

Environment setup and execution

```
python -m pip install -r requirements.txt
python -B -m src.train
```

These are commands for a compatible Python environment from the project root. python -m pip helps direct installation into the same interpreter selected by python. The verified environment uses Python 3.13.9; Python itself is not installed or pinned by requirements.txt. No installation or retraining was performed merely to author this guide.

Matplotlib and seaborn appear in the analytical notebook but are not required by the Phase 1 scripts, which do not draw plots. Jupyter is needed to interactively run the notebook but is not a runtime dependency of the pipeline. Streamlit is absent because the UI phase has not been implemented. PDF-authoring libraries are not part of the fraud pipeline requirements.

21 End-to-end data flow

Evidence: Workbook first data row; preprocessing functions; verified saved Random Forest inference.

Step 1: start with an actual transaction

The first workbook data row is a TRANSFER at step 1 with amount 181.00. It has origin balance 181.00 before and 0.00 after, and destination balances 0.00 before and after. Its provided label is isFraud=1. The predictor does not receive that label; it must make its own decision from the seven raw inputs.

Raw input	Value
step	1
amount	181.0
oldbalanceOrg	181.0
newbalanceOrig	0.0
oldbalanceDest	0.0
newbalanceDest	0.0
type	TRANSFER

Step 2: apply the exact arithmetic

```

orig_balance_change = 181.00 - 0.00 = 181.00
dest_balance_change = 0.00 - 0.00 = 0.00
orig_balance_error   = 181.00 - 181.00 = 0.00
dest_balance_error   = 181.00 - 0.00 = 181.00

```

Step 3: encode and align

The saved categories are declared before encoding. TRANSFER sets type_TRANSFER=True. type_CASH_OUT, type_DEBIT, and type_PAYMENT are False. The raw type string is replaced by these indicators. The table is then arranged into the saved order below.

Position	Feature	Value
1	step	1
2	amount	181.0
3	oldbalanceOrg	181.0
4	newbalanceOrig	0.0
5	oldbalanceDest	0.0
6	newbalanceDest	0.0

Position	Feature	Value
7	orig_balance_change	181.0
8	dest_balance_change	0.0
9	orig_balance_error	0.0
10	dest_balance_error	181.0
11	type_CASH_OUT	False
12	type_DEBIT	False
13	type_PAYMENT	False
14	type_TRANSFER	True

Step 4: apply the fitted forest

The model traverses its existing 100 trees using this feature vector. It does not examine the original Excel label during prediction and does not learn new tree structures. The verified returned dictionary is {"isFraud": 1, "fraud_probability": 0.97}. A caller can display "Fraud" for 1 or "Legitimate" for 0.

Excel row or input dictionary

|

v

Seven raw fields -> Four engineered values -> Four type indicators

|

v

14 columns in saved order -> Saved forest -> Class 1, score 0.97

What an output does and does not say

The predicted class agrees with the known label for this example. The probability is the model's class-1 score, not a verified probability of legal wrongdoing. One correct example is not an evaluation sample. The actual project-level evaluation remains the 2,229-row held-out test set documented earlier.

22 Reproducibility

Evidence: Phase 1 execution history; train.py guard; notebook saved outputs; saved artifact metadata.

A fresh Python process starts without the variables and fitted models left behind by earlier interactive notebook cells. Running the module from that clean state confirms it can load data, create features, split, fit, evaluate, and save on its own. This matters because notebooks can otherwise appear to work while relying on cells executed in a different order.

What was verified in Phase 1

- The complete training command ran in a fresh Anaconda Python 3.13.9 process.
- It produced 14 features, 8,913 training rows, and 2,229 testing rows.
- All three models matched the notebook metrics to 15 displayed decimal places.
- The Random Forest artifact was saved only after the reproduction check passed.
- Prediction preprocessing matched training preprocessing across the full workbook and one row from every transaction type.
- Saved-model prediction ran in a separate process with fit explicitly blocked during verification, confirming no retraining.
- Notebook and workbook hashes matched before and after Phase 1.

The process-run and blocked-fit checks are evidenced by the prior Phase 1 verification session. No standalone test suite or execution log was added to the project. The saved bundle independently preserves the resulting metric values and environment metadata; it does not record every verification step.

Version evidence

Component	Saved version
python	3.13.9
pandas	2.3.3
numpy	2.3.5
scipy	1.16.3
scikit-learn	1.7.2
openpyxl	3.1.5
joblib	1.5.2

Tolerance versus observed equality

train.py permits an absolute difference up to 0.000001 for each of the four metrics. The actual verified values agree to 15 displayed decimals, which is stronger than merely passing that tolerance. This check is specific to reproducing the known notebook on the known dataset. It is not evidence that any changed dataset should obtain the same scores.

What must remain consistent

Keep the workbook contents and row order, all four feature formulas, category reference and feature order, split settings, estimator settings, and compatible dependencies. Fixed seeds help control pseudo-random choices. They do not freeze every possible operating-system or numerical-library difference. The model artifact records full estimator defaults, so a future package upgrade can be compared with the actual training configuration.

The guide-authoring task did not rerun train.py or overwrite the model. It inspected the existing artifact and rechecked its saved prediction on the first workbook row. The notebook and source files remain the analytical and implementation evidence.

23 Current limitations

Evidence: Notebook time-step discussion and saved error; code and artifact inspection; supplied workbook metadata.

Unresolved time-step investigation

The notebook explicitly warns that step may reflect sampling or temporal separation. Legitimate records have step range 1-7; fraud records range 1-95. step remains in all three models, as required in Phase 1. No completed ablation without step, temporal holdout, or final decision resolving this concern is present. The high observed scores must be discussed with that qualification.

Scope of the results

The metrics describe one stratified random holdout of the supplied dataset. There is no external dataset evaluation, live transaction trial, cross-validation, or account-grouped evaluation recorded. The observed class balance is 10.25% fraud in this sample; the files do not establish a real production base rate. Test performance therefore does not prove equivalent performance on future data.

Financial and data interpretation

The notebook uses rupee labels, while the workbook does not establish currency provenance. Summed fraud amounts are transaction exposure, not measured net losses after recoveries. The notebook states step is a simulated hour, but no external source document or calendar mapping is supplied. The dataset's supplier, sampling process, and original fraud-labelling process cannot be determined from these files.

Prediction requirements and implementation boundaries

The saved predictor needs both pre- and post-transaction balances. A use case that scores before settlement would need to establish whether those after-balances are available at that point; the supplied project does not establish such availability. The function checks required fields and allowed types, but does not implement a complete form-validation layer, numeric business-rule validation, or handling for unseen transaction categories.

There is no probability calibration, custom cost-sensitive threshold, deployment monitoring, drift detection, or Streamlit interface in the implementation. These absences limit what can be claimed about a deployed system; they are not additional methods performed by the notebook. `predict_transaction` reloads the artifact on every call and has no cache.

Notebook and artifact reporting limits

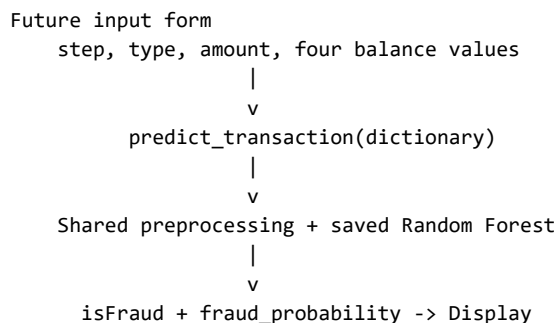
The original notebook still references an absolute Downloads path and contains a final plot error: `KeyError` for Logistic Regression. The model names are row values in `final_comparison`, but that plotting cell treats them as columns. Earlier comparison results exist successfully. The notebook was intentionally left unchanged; the working Python training command is separate.

Full-precision Gradient Boosting feature importance is not saved, so only its rounded notebook display is reported. Only Random Forest is persisted as a fitted estimator. The artifact records split settings but not explicit row indexes or source-data hashes. Reconstructing its evaluation requires the same original dataset and split procedure.

24 Streamlit: preparation for the next phase

Evidence: Conceptual future integration only. No Streamlit files or dependency are present.

Streamlit can provide a user interface around Python code. For this project, a future interface could collect transaction details, pass a dictionary to `predict_transaction`, and display the returned class and probability. The existing reusable preprocessing and saved model make that separation possible. This section is a design explanation, not a report that the interface has been built.



The seven form inputs

A future form would need numeric fields for `step`, `amount`, `oldbalanceOrg`, `newbalanceOrig`, `oldbalanceDest`, and `newbalanceDest`, plus a type selector using the five saved categories. It should not ask a user to invent engineered feature values, because preprocessing already calculates them. It should not ask for `isFraud` as an input to the model.

What stays outside the prediction request

Training would remain an explicit separate operation. The app would load the saved model state and use the same feature schema. It would need to communicate missing fields or invalid types clearly and explain the probability as a model score. A future implementation could consider caching the loaded artifact, but the present `predict.py` does not cache it. No UI, dependency changes, or deployment files were created for this guide.

25 Viva preparation: 90 questions

Evidence: Answers grounded in the inspected project; general concepts explain the implemented methods. Each item includes an oral answer, deeper explanation, and follow-up.

Practise the simple answer aloud first. Then use the deeper explanation if the examiner asks why or how. The follow-up is a prompt for further discussion, not a new project result. Questions 1-30 build foundations, 31-60 address implementation and models, and 61-90 examine interpretation and limitations.

Beginner: foundations and data

Q01. What is the objective of your project?

Say in the viva: I classify financial transactions as legitimate or fraudulent using the supplied transaction data.

Deeper explanation: The target is `isFraud`: 0 is legitimate and 1 is fraud. The project compares three supervised classifiers, then saves Random Forest for reusable prediction. The task uses provided labels; it does not establish how those labels were originally assigned.

Possible follow-up: Why is this supervised learning?

Q02. What is a DataFrame?

Say in the viva: A DataFrame is a table with labelled rows and columns in pandas.

Deeper explanation: The workbook becomes `df` after `pd.read_excel`. Each row represents a transaction, and each column holds a field such as amount. DataFrames support named column selection, arithmetic, filtering, grouping, and alignment by index.

Possible follow-up: How is a Series different from a DataFrame?

Q03. What is a feature?

Say in the viva: A feature is an input that a model uses to make a prediction.

Deeper explanation: This model receives 14 encoded features, including step, amount, balances, four derived balance quantities, and four type indicators. A raw workbook field and a final model feature are not always the same thing because encoding expands type.

Possible follow-up: Why are there 14 features but only 10 source columns?

Q04. What is the target?

Say in the viva: The target is the answer the model learns to predict: `isFraud`.

Deeper explanation: The target is copied into `y`. It is excluded from `X` so the model cannot read the answer directly. During training `y` supplies supervision; during testing it supplies the reference labels used to judge predictions.

Possible follow-up: What happens if you include `isFraud` in `X`?

Q05. What do 0 and 1 mean?

Say in the viva: Zero means legitimate and one means fraudulent.

Deeper explanation: The evaluation treats 1 as the positive class. Positive does not mean a good outcome: it is a naming convention. Precision and recall in the comparison table specifically describe detection of class 1.

Possible follow-up: Which class is the positive class in your confusion matrix?

Q06. How large is your dataset?

Say in the viva: It contains 11,142 transactions and 10 original columns.

Deeper explanation: The workbook has one worksheet named in. The first Excel row contains headers, so its used range reaches row 11,143. After preprocessing, the row count remains 11,142 but the feature matrix has 14 columns.

Possible follow-up: How many rows are used for testing?

Q07. What is EDA?

Say in the viva: EDA is examining the data to understand its structure, quality, and patterns before modelling.

Deeper explanation: The notebook checks dimensions, types, missing values, duplicates, class counts, transaction types, amounts, time steps, derived balances, and correlations. It also creates plots. These observations describe the dataset; they do not themselves train a classifier.

Possible follow-up: Which EDA result raised the strongest concern?

Q08. What does df.head() show?

Say in the viva: It shows the first five rows by default.

Deeper explanation: This is a loading and structure check. The first rows here happen to be fraud-labelled, but head is not a random sample and does not show the class balance of the whole dataset. value_counts gives that balance.

Possible follow-up: Why would using only head be misleading?

Q09. What does df.info() tell you?

Say in the viva: It shows column names, non-null counts, and data types.

Deeper explanation: The notebook reports two integer columns, five floating-point columns, and three object columns. Every column has 11,142 non-null values. Knowing type is text tells us it needs categorical encoding for these estimators.

Possible follow-up: Which object columns were excluded from modelling?

Q10. What does describe() calculate?

Say in the viva: It summarises numeric columns with counts, averages, spread, percentiles, and extremes.

Deeper explanation: The notebook transposes the result with .T. Mean can be pulled upward by large amounts, while median describes the middle sorted amount. Both are useful here because amounts are strongly spread and have a large upper tail.

Possible follow-up: Why compare mean and median?

Q11. Were there missing values?

Say in the viva: No missing values were found in any of the ten columns.

Deeper explanation: df.isnull().sum() counts missing cells by column. The project therefore does not impute missing values. This result applies to the supplied workbook and does not imply future submitted transactions will always contain valid values.

Possible follow-up: Does the prediction function validate every possible input problem?

Q12. Were there duplicate transactions?

Say in the viva: The notebook found zero exact duplicate rows.

Deeper explanation: `df.duplicated().sum()` compares complete rows. Two records with the same amount or a repeated account identifier are not necessarily duplicates. No duplicate-removal step was added because the actual full-row check found none.

Possible follow-up: Is a repeated account ID an exact duplicate row?

Q13. How common is fraud in the dataset?

Say in the viva: There are 1,142 fraud records, approximately 10.25% of all transactions.

Deeper explanation: The remaining 10,000 records are legitimate. This is the supplied sample's distribution, not a verified production fraud rate. The class imbalance is one reason to report recall and precision rather than accuracy alone.

Possible follow-up: Can a high-accuracy model still miss all fraud?

Q14. What transaction types exist?

Say in the viva: CASH_IN, CASH_OUT, DEBIT, PAYMENT, and TRANSFER.

Deeper explanation: The notebook observes fraud only in CASH_OUT and TRANSFER for this sample. There are 578 CASH_OUT fraud records and 564 TRANSFER fraud records. The other three categories have zero labelled fraud in the supplied data.

Possible follow-up: Does that prove PAYMENT can never be fraudulent?

Q15. How do you calculate fraud rate by type?

Say in the viva: I take the mean of the 0/1 fraud labels within each transaction type.

Deeper explanation: With binary labels, the sum counts fraud and the mean divides by all rows of the type. Multiplying by 100 gives a percentage. TRANSFER is 38.52% and CASH_OUT is 30.89%, rounded.

Possible follow-up: How does that differ from the share of all frauds belonging to a type?

Q16. What did you find about transaction amounts?

Say in the viva: Fraud-labelled transactions have larger mean and median amounts in this dataset.

Deeper explanation: The fraud mean is 1,192,628.93 versus 101,339.73 for legitimate transactions. Fraud median is 353,179.45 versus 12,725.24. These descriptive differences do not mean every large transaction is fraudulent.

Possible follow-up: Why is the mean much larger than the median?

Q17. What is feature engineering?

Say in the viva: It means creating useful model inputs from existing data.

Deeper explanation: This project derives origin and destination balance changes, then compares each change with amount. The formulas are deterministic per transaction. They expose arithmetic relationships already present in the raw fields rather than adding new external information.

Possible follow-up: Which four engineered features did you use?

Q18. What is orig_balance_change?

Say in the viva: It is `oldbalanceOrig` minus `newbalanceOrig`.

Deeper explanation: It measures the recorded decrease in the origin balance. A positive value indicates a decrease and a negative value an increase. The first workbook row gives 181 minus 0, equal to 181.

Possible follow-up: Why is the subtraction direction important?

Q19. What is dest_balance_change?

Say in the viva: It is `newbalanceDest` minus `oldbalanceDest`.

Deeper explanation: It measures the recorded increase in the destination balance. Its subtraction direction is the opposite of origin change because it is intended to describe receiving-side growth. The first row gives zero because both destination balances are zero.

Possible follow-up: What does a negative destination change mean?

Q20. What is orig_balance_error?

Say in the viva: It is `amount` minus `orig_balance_change`.

Deeper explanation: It compares the transaction amount with the origin decrease. Zero means the two agree; it is not a guarantee of legitimacy. Fraud has a median of zero for this feature in the notebook's group summary.

Possible follow-up: Can a zero balance error occur in fraud?

Q21. What is dest_balance_error?

Say in the viva: It is `amount` minus `dest_balance_change`.

Deeper explanation: It compares amount with the destination increase. In the first row, amount is 181 and destination change is zero, giving 181. The word error labels a residual, not an automatically confirmed mistake or fraud.

Possible follow-up: Why should you avoid turning nonzero error directly into a fraud rule?

Q22. Why exclude account identifiers?

Say in the viva: Their numbers are identifiers, not meaningful ordered measurements.

Deeper explanation: The notebook selects a feature list that excludes `nameOrig` and `nameDest`. It does not create account-history features or identity encodings. Simply treating digits inside an ID as magnitude could introduce meaningless ordering.

Possible follow-up: Could a different future project use account history?

Q23. What is one-hot encoding?

Say in the viva: It creates an indicator column for each category.

Deeper explanation: Here `pd.get_dummies` encodes `type` and drops the first category. Four Boolean columns remain. This lets the models use category membership numerically without assuming TRANSFER is a larger numeric quantity than PAYMENT.

Possible follow-up: Which category is represented by all zero indicators?

Q24. What does `drop_first=True` do?

Say in the viva: It removes one category indicator, leaving a reference category.

Deeper explanation: CASH_IN is the reference in the saved sorted category order. All four remaining indicators are False for CASH_IN. The rows are not deleted; only the redundant indicator column is omitted.

Possible follow-up: How many indicators remain from five categories?

Q25. What are X and y?

Say in the viva: X contains the model inputs and y contains the correct fraud labels.

Deeper explanation: X is a 11,142-by-14 DataFrame after encoding, while y is a Series of 11,142 labels. Their row alignment is essential because each transaction must stay paired with its own answer.

Possible follow-up: What could happen if you shuffled only y?

Q26. Why use a train/test split?

Say in the viva: To evaluate the model on rows it did not use for fitting.

Deeper explanation: The training set teaches the model and the held-out set measures its predictions. Testing on training rows would not answer the same generalisation question. This project uses one stratified random holdout rather than cross-validation.

Possible follow-up: What is the exam-practice analogy?

Q27. What split did you use?

Say in the viva: Eighty percent for training and twenty percent for testing.

Deeper explanation: The actual counts are 8,913 and 2,229 because rows are indivisible. The test set contains 2,001 legitimate records and 228 fraud records. All three estimators share exactly this split.

Possible follow-up: Why should model comparisons use the same split?

Q28. What is stratification?

Say in the viva: It keeps class proportions approximately similar in train and test.

Deeper explanation: `stratify=y` uses the target labels to guide allocation. It does not rebalance the classes to equal sizes and does not generate synthetic data. Both partitions retain approximately the original fraud fraction.

Possible follow-up: How is stratification different from oversampling?

Q29. What does `random_state=42` mean?

Say in the viva: It fixes a random seed so the same procedure can be reproduced.

Deeper explanation: The seed controls pseudo-random choices where the algorithm uses them. It is used in the split and model constructors. It does not guarantee all software versions produce identical results or improve accuracy just because the number is 42.

Possible follow-up: Does Logistic Regression's default solver use this seed in the same way as Random Forest?

Q30. What is the difference between training and prediction?

Say in the viva: Training learns model parameters; prediction applies already learned parameters.

Deeper explanation: `train.py` calls `fit` on labelled training data. `predict.py` loads the existing fitted forest and calls `predict` and `predict_proba`. It does not read the dataset or refit the model for each user transaction.

Possible follow-up: Why would retraining on every request be undesirable?

Intermediate: models and evaluation

Q31. What is Logistic Regression?

Say in the viva: It is a classifier that converts a weighted feature score into a probability.

Deeper explanation: A sigmoid maps a linear score into the interval from zero to one. Despite regression in its name, the estimator is used here to classify legitimate versus fraudulent transactions. The project uses the default lbfgs solver with L2 regularisation.

Possible follow-up: What is the sigmoid formula?

Q32. What does `max_iter=1000` control?

Say in the viva: It is the maximum number of optimisation iterations for Logistic Regression.

Deeper explanation: It is a cap, not a guarantee that exactly 1,000 iterations run. It does not specify the number of examples or a decision threshold. The notebook provides this setting and Phase 1 preserves it.

Possible follow-up: Does increasing `max_iter` necessarily improve test accuracy?

Q33. How did Logistic Regression perform?

Say in the viva: It had perfect precision but detected 205 of the 228 frauds.

Deeper explanation: It missed 23 fraud-labelled test rows and made no false-positive predictions. Recall was 0.899123, F1 0.946882, and ROC-AUC 0.991154. Perfect precision therefore did not mean perfect recall.

Possible follow-up: Which denominator gives recall?

Q34. What is a decision tree?

Say in the viva: It predicts by following a sequence of learned feature conditions.

Deeper explanation: Each split routes a row to a branch and eventually to a leaf. The leaf's training labels determine class proportions. Both tree ensembles in this project use trees, but combine and train them differently.

Possible follow-up: What is a leaf node?

Q35. What is Random Forest?

Say in the viva: It combines many decision trees trained with randomness.

Deeper explanation: The project uses 100 trees. Bootstrap sampling and random feature consideration diversify them. Their class probabilities are averaged in this scikit-learn implementation, then the class with the largest probability is selected.

Possible follow-up: Why can averaging help compared with one tree?

Q36. What is ensemble learning?

Say in the viva: It combines multiple models into one predictor.

Deeper explanation: Random Forest combines varied trees, while Gradient Boosting builds successive correction trees. Both are ensembles but do not train their components in the same way. This project compares their held-out metrics without claiming one family is always superior.

Possible follow-up: How do bagging and boosting differ conceptually?

Q37. What does `n_estimators=100` mean?

Say in the viva: It sets 100 trees in Random Forest and 100 boosting stages in this binary Gradient Boosting model.

Deeper explanation: It is a model-size parameter. It is unrelated to the number of input columns or the percentage of data used for training. In the binary boosting configuration here, each stage fits one tree.

Possible follow-up: Is a boosting stage the same as tree depth?

Q38. What does `n_jobs=-1` mean?

Say in the viva: It requests all available processors for supported parallel work.

Deeper explanation: Random Forest uses this setting for supported operations across trees. It does not create an unlimited number of trees or change the chosen fraud threshold. The project preserves it exactly from the notebook.

Possible follow-up: Why did the sandbox initially block training?

Q39. Why was special permission needed on Windows?

Say in the viva: The execution sandbox blocked Windows worker infrastructure used by parallel Random Forest operations.

Deeper explanation: The traceback showed a `WinError 5` during `ThreadPool` and pipe setup. An approved run outside that sandbox succeeded without changing `n_jobs`. This is a property of that restricted execution environment, not a general Windows administrator requirement.

Possible follow-up: Did you change the methodology to work around the error?

Q40. What is bootstrap sampling?

Say in the viva: It samples training rows with replacement for individual trees.

Deeper explanation: With replacement means a row may be drawn repeatedly and another may not be drawn for a given tree. The forest's saved default `bootstrap=True` uses this mechanism. The held-out test rows still are not used in fitting.

Possible follow-up: Are bootstrap samples the same as the train/test split?

Q41. What does `max_features="sqrt"` mean?

Say in the viva: It limits the candidate features considered at tree splits using the square-root setting.

Deeper explanation: The saved forest inherits this default. Random feature consideration helps trees differ from one another. It does not remove columns from the 14-feature input schema or reduce the complete forest to only a few usable features.

Possible follow-up: How is this different from permanently dropping a feature?

Q42. How does the forest return probability?

Say in the viva: It averages the class probabilities from its fitted trees.

Deeper explanation: A tree uses the training class proportions in the reached leaf. The forest averages those outputs across trees. Therefore the committee-vote analogy should not be interpreted as necessarily counting only hard class votes.

Possible follow-up: Does probability 0.97 prove a calibrated 97% real-world fraud chance?

Q43. What is Gradient Boosting?

Say in the viva: It builds trees sequentially to improve the current combined model.

Deeper explanation: Later trees fit correction signals derived from the loss gradient. This differs from training independent trees and averaging them. The project uses `log_loss` by default, 100 stages, depth 3, and learning rate 0.1.

Possible follow-up: Why is it called gradient boosting?

Q44. What does `learning_rate=0.1` do?

Say in the viva: It scales each new boosting tree's contribution.

Deeper explanation: A smaller learning rate makes each update more conservative. It interacts with the number of stages, but no search over those settings was performed here. The value is not a fraud-rate estimate or a test fraction.

Possible follow-up: What might happen if learning rate changed but stage count stayed fixed?

Q45. What does `max_depth=3` mean?

Say in the viva: It limits the depth of each Gradient Boosting tree.

Deeper explanation: Depth controls how many successive conditions a component tree can apply. The complete ensemble still has 100 stages. The Random Forest has a different default, `max_depth=None`, which the project does not override.

Possible follow-up: Does depth 3 mean there are only three trees?

Q46. Did you scale the features?

Say in the viva: No. I preserved the notebook's unscaled feature inputs.

Deeper explanation: Neither `StandardScaler` nor another normalisation step appears in preprocessing. This is especially relevant when describing Logistic Regression: the guide must not imply scaling was used just because it is common in other projects.

Possible follow-up: Why must a reproduction task avoid silently adding preprocessing?

Q47. Did you balance the classes with SMOTE or class weights?

Say in the viva: No. The project uses stratification but no resampling or class weighting.

Deeper explanation: The class distribution stays imbalanced. The stored estimators use `class_weight=None` where applicable, and no synthetic-sampling package is included. Stratification helps preserve proportions between partitions but does not equalise classes.

Possible follow-up: Why are precision and recall still useful without resampling?

Q48. What is a confusion matrix?

Say in the viva: It counts correct and incorrect predictions for each class.

Deeper explanation: With actual labels in rows and predicted labels in columns, this binary matrix is `[[TN, FP], [FN, TP]]`. Random Forest's matrix is `[[2001, 0], [1, 227]]`. It provides the counts used in several metrics.

Possible follow-up: Where is the missed-fraud count in that layout?

Q49. What is a true positive here?

Say in the viva: A fraudulent transaction correctly predicted as fraud.

Deeper explanation: The test set has 227 true positives for Random Forest. These form the numerator of both precision and recall. The two metrics differ because their denominators ask different questions.

Possible follow-up: Why can precision be 1 while recall is below 1?

Q50. What is a true negative here?

Say in the viva: A legitimate transaction correctly predicted as legitimate.

Deeper explanation: There are 2,001 true negatives for Random Forest. They contribute to accuracy and the false-positive-rate denominator. They do not appear directly in the formula for fraud-class F1.

Possible follow-up: Which common metric includes true negatives in its numerator?

Q51. What is a false positive?

Say in the viva: A legitimate transaction incorrectly flagged as fraud.

Deeper explanation: Random Forest has zero false positives on this test set. In a future business workflow false alerts could require investigation, but this project does not measure analyst time or customer disruption costs.

Possible follow-up: What does a false-positive transaction amount measure?

Q52. What is a false negative?

Say in the viva: A fraudulent transaction incorrectly predicted as legitimate.

Deeper explanation: Random Forest misses one fraud-labelled test transaction. Its recorded amount is 416,001.33. This connects recall to the notebook's amount-based potential-loss calculation.

Possible follow-up: Does one false negative always mean a small financial exposure?

Q53. What is precision?

Say in the viva: It is the fraction of predicted fraud alerts that are actually fraud.

Deeper explanation: Precision equals TP divided by TP plus FP. For Random Forest it is 227 divided by 227, or 1.0. The denominator is predicted positives, not all actual frauds and not every test transaction.

Possible follow-up: How would one false positive affect the denominator?

Q54. What is recall?

Say in the viva: It is the fraction of actual fraud cases that the model detects.

Deeper explanation: Recall equals TP divided by TP plus FN. For Random Forest it is 227/228, approximately 99.56%. It measures coverage of fraud-labelled rows and helps reveal what a precise but conservative model misses.

Possible follow-up: What was Logistic Regression recall?

Q55. What is F1?

Say in the viva: It is the harmonic mean of precision and recall.

Deeper explanation: The equivalent count formula is $2TP/(2TP+FP+FN)$. Random Forest gives 454/455, or 0.997802. It balances two error perspectives but does not incorporate the monetary value of individual transactions.

Possible follow-up: Why is F1 not a financial-cost metric?

Q56. What is accuracy?

Say in the viva: It is the fraction of all test decisions that are correct.

Deeper explanation: Random Forest accuracy is $(2001+227)/2229$, approximately 99.9551%. It includes both classes. Because legitimate transactions are the majority, accuracy alone can hide poor detection of the minority fraud class.

Possible follow-up: How accurate would an always-legitimate teaching baseline be?

Q57. What is ROC?

Say in the viva: It is a curve of true-positive rate against false-positive rate across thresholds.

Deeper explanation: TPR is recall, $TP/(TP+FN)$, and FPR is $FP/(FP+TN)$. Different score thresholds produce different points. The notebook computes the area under this curve but does not actually draw an ROC plot.

Possible follow-up: Why is one confusion matrix insufficient to describe the whole curve?

Q58. What is ROC-AUC?

Say in the viva: It measures how well fraud scores rank fraud above legitimate cases across thresholds.

Deeper explanation: It is the area under ROC and can be interpreted using positive-negative score pairs with half credit for ties. Random Forest's value is 0.9999572582129987, not exactly 1 despite rounded display.

Possible follow-up: Why should AUC use probabilities rather than only class labels?

Q59. Can two models share F1 but have different AUC?

Say in the viva: Yes. Their class decisions can match in aggregate while probability rankings differ.

Deeper explanation: Random Forest and Gradient Boosting share precision, recall, F1, and confusion-matrix counts here. Their AUC values are 0.999957 and 0.999678. AUC uses more information than one set of hard labels.

Possible follow-up: Does an identical confusion matrix prove identical scores?

Q60. Which model did you save and why?

Say in the viva: I saved Random Forest, following the notebook's business analysis and Phase 1 specification.

Deeper explanation: It has the numerically highest AUC in this split and ties Gradient Boosting on the other displayed metrics. There is no universal best-model proof or significance test. The saved forest remains fitted on the training partition.

Possible follow-up: Was it refitted on the entire dataset before saving?

Advanced: implementation, interpretation, and defence

Q61. What is feature importance?

Say in the viva: It measures how useful a feature was to the fitted trees under their split-importance calculation.

Deeper explanation: The project uses impurity-based feature_importances_. Values are normalised within a model. They describe this fitted model and dataset, not causal effects. Random Forest assigns the largest importance to step.

Possible follow-up: Can a feature's importance be interpreted as a probability of fraud?

Q62. Which Random Forest features are most important?

Say in the viva: step, orig_balance_change, orig_balance_error, newbalanceOrig, and amount are the top five.

Deeper explanation: Their approximate shares are 44.60%, 14.11%, 12.35%, 7.03%, and 6.58%. These percentages refer to the importance measure, not shares of fraud cases or individual prediction confidence.

Possible follow-up: Why should step's large share be discussed carefully?

Q63. Does step cause fraud?

Say in the viva: Feature importance does not establish causation.

Deeper explanation: The sample places legitimate rows only in steps 1-7 and fraud rows in steps 1-95. A model can exploit this association. The notebook raises a sampling or temporal concern and leaves its investigation unresolved.

Possible follow-up: Was step removed in Phase 1?

Q64. Why did you keep step?

Say in the viva: Phase 1 was a faithful reproduction, and the notebook had not decided to remove it.

Deeper explanation: All three notebook models include step. Removing it would produce a new experiment with potentially different results. The limitation is documented, and no alternative no-step metrics are invented.

Possible follow-up: Would a future no-step comparison replace the original results or be a separate experiment?

Q65. Do you have exact Gradient Boosting importance values?

Say in the viva: Only the notebook's values rounded to two decimal places are saved.

Deeper explanation: The artifact contains fitted Random Forest only. Gradient Boosting's notebook table displays step as 0.93, newbalanceOrig as 0.05, and orig_balance_change as 0.01. A displayed 0.00 is not proof of exact zero.

Possible follow-up: Why must you avoid inventing extra decimal places?

Q66. How did you calculate total fraud amount?

Say in the viva: I summed amount for every row with isFraud equal to 1.

Deeper explanation: This full-dataset sum is 1,361,982,240.43 over 1,142 rows. It is distinct from test-set missed-fraud exposure. The column records transaction amounts, not recoveries or reconciled realised losses.

Possible follow-up: What is the scope of the missed-fraud calculation?

Q67. How did you calculate missed-fraud exposure?

Say in the viva: I selected test rows with actual fraud and predicted legitimate, then summed their amounts.

Deeper explanation: The Boolean mask identifies false negatives. `y_test` retains original row indexes, which `df.loc` uses to recover matching amount values. The one selected row has amount 416,001.33.

Possible follow-up: Why is index alignment important after splitting?

Q68. Why does financial impact matter?

Say in the viva: A small error count can still involve a large transaction amount.

Deeper explanation: Random Forest misses only one fraud in the test set, but its amount is 416,001.33. Metrics count cases equally, while an amount sum highlights exposure differences. No wider cost optimisation is implemented here.

Possible follow-up: Could two models with the same recall have different missed amounts?

Q69. Are the amounts definitely Indian rupees?

Say in the viva: The notebook labels them with a rupee symbol, but the workbook does not establish currency provenance.

Deeper explanation: The guide preserves the recorded values and identifies that interpretation limit. A display symbol does not independently prove the dataset's monetary unit or convert one currency to another.

Possible follow-up: How should you phrase the financial result in a viva?

Q70. What is joblib used for?

Say in the viva: It saves and loads the fitted model bundle.

Deeper explanation: `joblib.dump` serialises the forest and metadata after successful training. `joblib.load` restores them for prediction in a fresh process. The artifact contains Python objects and should only be loaded from a trusted source.

Possible follow-up: Is the joblib file just a table of metric values?

Q71. What is stored with the model?

Say in the viva: The preprocessing schema, evaluations, model settings, split settings, and software versions.

Deeper explanation: The schema includes category order, feature order, dtypes, raw field names, encoding name, and `drop_first`. The bundle does not contain raw data, fitted alternative models, or a serialised copy of the preprocessing formulas.

Possible follow-up: Why must preprocessing.py remain compatible with the artifact?

Q72. Why is the schema important?

Say in the viva: It makes future inputs match the representation used during training.

Deeper explanation: Models interpret named columns and their ordering consistently. Saved categories prevent single-row dummy encoding from losing necessary columns. Stored feature columns provide a concrete alignment contract for prediction.

Possible follow-up: What could go wrong if amount were supplied in a balance column's position?

Q73. Why not just call `get_dummies` on one new row?

Say in the viva: It could see only one category and drop its only indicator.

Deeper explanation: With `drop_first=True`, category discovery from a single TRANSFER row would not recreate the training schema. `prepare_prediction_data` first declares all saved categories, then encodes and reindexes columns.

Possible follow-up: How does `pd.Categorical` solve that problem?

Q74. What happens for an unknown transaction type?

Say in the viva: The prediction preprocessing raises `ValueError`.

Deeper explanation: `isin` checks every provided type against the saved category list. Unknown or missing types are rejected instead of silently becoming an all-zero indicator vector that would look like CASH_IN. No unknown-category fallback is implemented.

Possible follow-up: Where would a future interface display this error?

Q75. Does prediction retrain the model?

Say in the viva: No. It loads the saved forest and calls its prediction methods.

Deeper explanation: There is no `fit` call in `predict.py`. Phase 1 also verified prediction while `fit` was explicitly blocked in the verification process. The source has no dependency on the training entry point for ordinary inference.

Possible follow-up: Does prediction need the Excel workbook?

Q76. What does `predict_transaction` return?

Say in the viva: A dictionary containing `isFraud` and `fraud_probability`.

Deeper explanation: `isFraud` is a normal Python integer. `fraud_probability` is a Python float from the probability column corresponding to class 1. The function locates that column through `model.classes_` rather than leaving the label interpretation implicit.

Possible follow-up: How would you display the result to a nontechnical user?

Q77. Why use a project-relative path?

Say in the viva: It avoids depending on one user's Downloads folder or current directory.

Deeper explanation: `Path(__file__).resolve().parents[1]` locates the project root relative to `preprocessing.py`. The workbook path is then joined to that root. This supports moving the project as a unit while preserving its directory structure.

Possible follow-up: Would moving `preprocessing.py` to a different depth require a path change?

Q78. What is `requirements.txt`?

Say in the viva: It lists the package versions used by the implementation.

Deeper explanation: It pins `pandas`, `scikit-learn`, `openpyxl`, `joblib`, `NumPy`, and `SciPy`. The numerical dependencies are pinned even when not directly imported. Python itself and every possible transitive package are not locked by this file.

Possible follow-up: Why can two Python installations on the same computer behave differently?

Q79. Why are NumPy and SciPy included?

Say in the viva: They are necessary numerical dependencies of the machine-learning stack.

Deeper explanation: The project modules do not directly import them, but pandas and scikit-learn rely on numerical-array and scientific-computing functionality. Explicit versions document the numerical environment that reproduced the notebook.

Possible follow-up: Why is seaborn absent from the pipeline requirements?

Q80. What does a fresh process verify?

Say in the viva: It shows the pipeline works without leftover notebook variables.

Deeper explanation: A clean process must load the workbook, create all features, instantiate and fit models, and produce outputs itself. Phase 1 succeeded in the Anaconda environment and matched the notebook metrics to 15 displayed decimals.

Possible follow-up: Why can notebook execution order create hidden dependencies?

Q81. What if the new metrics differ?

Say in the viva: train.py reports discrepancies and stops before saving a new artifact.

Deeper explanation: The guard compares each metric with the notebook reference using absolute tolerance 0.000001. It does not tune parameters until results match. An existing artifact remains on disk if a later training run fails the check.

Possible follow-up: Why should an old artifact not be mistaken for a successful new run?

Q82. Why are seeds not enough for reproducibility?

Say in the viva: The data, feature order, settings, and software environment must also be consistent.

Deeper explanation: A seed controls pseudo-random choices, but changing row order, library behaviour, formulas, or category ordering can change outputs. The bundle records versions and full estimator parameters to make the experiment more inspectable.

Possible follow-up: Does the artifact store the original data hash?

Q83. Is the pipeline a scikit-learn Pipeline object?

Say in the viva: No. It is a sequence of reusable Python functions and estimator calls.

Deeper explanation: preprocessing.py handles transformations, train.py orchestrates fitting and evaluation, and predict.py applies the saved model. The word pipeline describes the workflow; no sklearn.pipeline.Pipeline instance is constructed.

Possible follow-up: Where are the feature formulas stored?

Q84. What preprocessing validation is present?

Say in the viva: It checks required raw fields and allowed transaction types.

Deeper explanation: It also aligns encoded columns and restores dummy dtypes. It does not implement a complete numeric business-validation layer. Arithmetic and the estimator may reject invalid values, but a future UI should not assume every bad input has a tailored message.

Possible follow-up: Does it reject every negative balance by an explicit rule?

Q85. What does deployment mean for this project?

Say in the viva: It would mean making the saved predictor available through an application or service.

Deeper explanation: Phase 1 provides reusable inference but no running application. Deployment would need input collection, validation, compatible dependencies, artifact access, and clear output interpretation. No live performance evidence is currently available.

Possible follow-up: Which parts are already reusable by a future interface?

Q86. How would Streamlit use the model?

Say in the viva: It would collect seven raw fields, call `predict_transaction`, and display the returned result.

Deeper explanation: The interface should reuse preprocessing rather than ask users to calculate engineered values. Training would remain separate. No Streamlit app, dependency, cached loader, or deployment configuration exists in the current project.

Possible follow-up: Why should Streamlit not retrain on every interaction?

Q87. Can this model necessarily score before a transaction completes?

Say in the viva: The current inputs include post-transaction balances, so that availability must first be established.

Deeper explanation: `newbalanceOrig` and `newbalanceDest` are required by the formulas. The files do not demonstrate an operational pre-authorisation data feed with those values available. The guide therefore does not claim the model is a ready pre-transaction prevention system.

Possible follow-up: How is post-event review different from pre-event approval?

Q88. What limitations should you state first?

Say in the viva: The unresolved step pattern and the fact that evaluation uses only this dataset's holdout.

Deeper explanation: Currency provenance is also unresolved, and no external evaluation or live monitoring is present. State limitations supported by the files rather than inventing additional experiments or claiming the sample proves production reliability.

Possible follow-up: Why does high test performance not remove the need to discuss sampling?

Q89. What error remains in the original notebook?

Say in the viva: Its final comparison line plot raises `KeyError` for Logistic Regression.

Deeper explanation: That cell treats model names as columns, but `final_comparison` stores them as values in the `Model` column. Earlier tables and a bar chart exist successfully. The original notebook was deliberately left unchanged during Phase 1.

Possible follow-up: Does that plotting error invalidate the separately saved model metrics?

Q90. Give a one-minute explanation of the whole project.

Say in the viva: I analysed 11,142 transactions, engineered four balance features, encoded type, and compared three classifiers on a stratified 80/20 split.

Deeper explanation: I preserved the notebook in a reusable Python implementation. Random Forest was saved with its schema. It detected 227 of 228 test frauds with no false positives, and the missed amount was 416,001.33. The step concern remains unresolved, and Streamlit is future work.

Possible follow-up: Which claim in that explanation is limited to the held-out test set?

26 Quick revision

Evidence: Condensed review of the project evidence and explanations in sections 1-25.

Definitions and vocabulary

Term	Meaning to remember
Feature / target	Input evidence / known answer to predict.
DataFrame / Series	Labelled table / labelled one-dimensional column.
Classification / regression	Predict a category / predict a numeric quantity; Logistic Regression is used as a classifier.
Fit / inference	Learn from labelled training data / apply a fitted model.
Encoding / schema	Convert categories to numeric indicators / preserve the representation contract.
Stratification / seed	Keep class proportions / fix pseudo-random choices.
Ensemble / bootstrap	Combine component models / sample with replacement.
Hyperparameter / learned parameter	Setting supplied before fitting / state learned through fitting.
Threshold / probability	Rule converting a score to a decision / model-estimated class score.
Support / macro / weighted	Actual class count / equal-class average / average weighted by support.
Serialization / artifact	Store object state / saved model-and-metadata bundle.
Correlation / causation	Association / one variable producing a change in another; this project does not establish causation.

Formulas

```
orig_balance_change = oldbalanceOrig - newbalanceOrig
dest_balance_change = newbalanceDest - oldbalanceDest
orig_balance_error  = amount - orig_balance_change
dest_balance_error  = amount - dest_balance_change
```

```
Accuracy  = (TP + TN) / N
Precision = TP / (TP + FP)
Recall    = TP / (TP + FN)
F1        = 2*TP / (2*TP + FP + FN)
FPR       = FP / (FP + TN)
N         = TP + TN + FP + FN
Fraud rate by type = fraud count of type / all rows of type
Missed-fraud amount = sum(amount for test false negatives)
```

Numbers and settings

Item	Value
Rows / source columns / model features	11,142 / 10 / 14
Legitimate / fraud	10,000 / 1,142
Train / test	8,913 / 2,229
Test legitimate / fraud	2,001 / 228
RF TN / FP / FN / TP	2,001 / 0 / 1 / 227
RF precision / recall / F1	1.000000 / 0.995614 / 0.997802
RF ROC-AUC / accuracy	0.999957 / 99.955137%
Full-data fraud amount / test missed amount	1,361,982,240.43 / 416,001.33
Split	test_size=0.20, random_state=42, stratify=y
Logistic Regression	max_iter=1000, random_state=42
Random Forest	n_estimators=100, random_state=42, n_jobs=-1
Gradient Boosting	n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42

Important functions

Function or method	What it does here
pd.read_excel / df.shape / df.info	Load, count dimensions, inspect structure.
describe / isnull / duplicated	Summarise numbers, detect missing cells, detect repeated rows.
value_counts / crosstab / groupby	Count labels, cross-tabulate categories, aggregate groups.
pd.get_dummies / pd.Categorical / reindex	Encode, declare saved categories, enforce feature order.
train_test_split / fit	Create holdout partitions / learn model state.
predict / predict_proba	Return class labels / per-class probability estimates.
precision_score / recall_score / f1_score	Measure fraud alert quality and detection coverage.
roc_auc_score / confusion_matrix	Measure score ranking / count decision outcomes.
classification_report / get_params	Collect class metrics and accuracy / record estimator settings.
joblib.dump / joblib.load	Persist / restore the bundle.

Twenty questions to rehearse immediately before the viva

1. What is the objective of your project?

I classify financial transactions as legitimate or fraudulent using the supplied transaction data.

2. How large is your dataset?

It contains 11,142 transactions and 10 original columns.

3. How common is fraud in the dataset?

There are 1,142 fraud records, approximately 10.25% of all transactions.

4. What is feature engineering?

It means creating useful model inputs from existing data.

5. What is orig_balance_change?

It is oldbalanceOrg minus newbalanceOrig.

6. What is dest_balance_change?

It is newbalanceDest minus oldbalanceDest.

7. What is orig_balance_error?

It is amount minus orig_balance_change.

8. What is dest_balance_error?

It is amount minus dest_balance_change.

9. What does drop_first=True do?

It removes one category indicator, leaving a reference category.

10. What split did you use?

Eighty percent for training and twenty percent for testing.

11. What is stratification?

It keeps class proportions approximately similar in train and test.

12. What does random_state=42 mean?

It fixes a random seed so the same procedure can be reproduced.

13. What is Random Forest?

It combines many decision trees trained with randomness.

14. What does n_jobs=-1 mean?

It requests all available processors for supported parallel work.

15. What is Gradient Boosting?

It builds trees sequentially to improve the current combined model.

16. What is a confusion matrix?

It counts correct and incorrect predictions for each class.

17. What is precision?

It is the fraction of predicted fraud alerts that are actually fraud.

18. What is recall?

It is the fraction of actual fraud cases that the model detects.

19. What is ROC-AUC?

It measures how well fraud scores rank fraud above legitimate cases across thresholds.

20. What limitations should you state first?

The unresolved step pattern and the fact that evaluation uses only this dataset's holdout.

Source map and technical references

Primary project sources: original notebook cells are numbered in file order from 1, not by execution counter; workbook sheet in holds A1:J11143; source-code line numbers refer to the inspected Phase 1 files.

Evaluation and version values were read from `models/random_forest.joblib`. Requirements were read directly from `requirements.txt`. The earlier Phase 1 conversation supplies the fresh-process and Windows sandbox verification history.

General API details were checked against the official scikit-learn 1.7 references linked in the model and metric sections. These references explain library behaviour; they are not sources of this project's dataset counts or metrics. All project-specific outputs come from the supplied files or explicitly identified verification history.

27 Final project cheat sheet

Evidence: Two-page pre-viva summary. Read page 1 for methodology and page 2 for results and implementation.

Page 1 of 2: data and method

One-sentence objective: Classify financial transactions as legitimate (isFraud=0) or fraud (isFraud=1), using the supplied workbook and the notebook's unchanged methodology.

Data and split	Remember
Dataset	Fraud_Analysis_Dataset.xlsx; sheet in; 11,142 rows; 10 original columns.
Class distribution	10,000 legitimate; 1,142 fraud; fraud share 10.25%.
Quality	No missing values; no exact duplicate rows.
Type categories	CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER.
Raw model fields	step, type, amount, oldbalanceOrig, newbalanceOrig, oldbalanceDest, newbalanceDest.
Excluded	nameOrig, nameDest; isFraud is y, never an X feature.
Encoding	get_dummies(type, drop_first=True); CASH_IN baseline; four dummy columns; 14 final features.
Split	80/20; stratify=y; random_state=42; train 8,913; test 2,229.

Origin change = oldbalanceOrig - newbalanceOrig
 Destination change = newbalanceDest - oldbalanceDest
 Origin error = amount - origin change
 Destination error = amount - destination change

Model	Exact explicit settings
Logistic Regression	max_iter=1000, random_state=42
Random Forest	n_estimators=100, random_state=42, n_jobs=-1
Gradient Boosting	n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42

Concepts: Logistic Regression maps a weighted score to a probability. Random Forest averages varied trees' class probabilities. Gradient Boosting adds trees sequentially to correct the current model's loss. n_jobs=-1 controls parallel execution. No scaling, resampling, new models, or removal of step occurred.

Final cheat sheet: results and defence

Page 2 of 2

Model	Precision	Recall	F1	ROC-AUC
Logistic Regression	1.000000	0.899123	0.946882	0.991154
Random Forest	1.000000	0.995614	0.997802	0.999957
Gradient Boosting	1.000000	0.995614	0.997802	0.999678

RF confusion-matrix fact	Interpretation
TN=2,001; FP=0	All legitimate test transactions classified correctly.
FN=1; TP=227	227 of 228 fraud cases detected; one missed.
Accuracy=2,228/2,229	99.955137%, rounded to 99.96%.

Precision = $TP / (TP + FP) = 1.0$

Recall = $TP / (TP + FN) = 227 / 228$

F1 = $2TP / (2TP + FP + FN) = 454 / 455$

ROC: TPR versus FPR across thresholds

AUC: ranking quality, not classification accuracy

Financial scope: Full dataset: 1,142 frauds totalling 1,361,982,240.43. RF test errors: one missed fraud with amount 416,001.33; zero false positives and amount 0.00. Rupee notation appears in the notebook, but currency provenance is unconfirmed.

Files: preprocessing.py loads and transforms; train.py trains/evaluates/saves; predict.py loads and predicts without fitting; random_forest.joblib contains the fitted forest, schema, evaluations, parameters, split settings, and versions; requirements.txt pins six packages.

Verified: Fresh-process training matched notebook metrics to 15 displayed decimals. Saved-model prediction works for all five categories. Example first row: TRANSFER, amount 181, origin 181→0, destination 0→0; result class 1, score 0.97. Run training as `python -B -m src.train` in the compatible environment.

Defend the scope: step remains unresolved: legitimate steps 1-7, fraud steps 1-95. RF step importance ≈44.6%; GB notebook display ≈0.93. Importance is not causation. ROC-AUC rounds to 1.0000 but is below 1. Test performance is dataset-specific. Streamlit is future work.

Closing viva answer: “I reproduced the notebook in reusable Python, preserved its data and settings, and saved Random Forest with its feature schema. The evaluation is strong on this held-out sample, and I explicitly retain the unresolved step concern and the distinction between transaction exposure and realised loss.”